

Federated Learning

Ambient Intelligence and Domotics

Master Degree: Artificial Intelligence for Science and Technology

Gabriele Civitarese

EveryWare Lab, Department of Computer Science

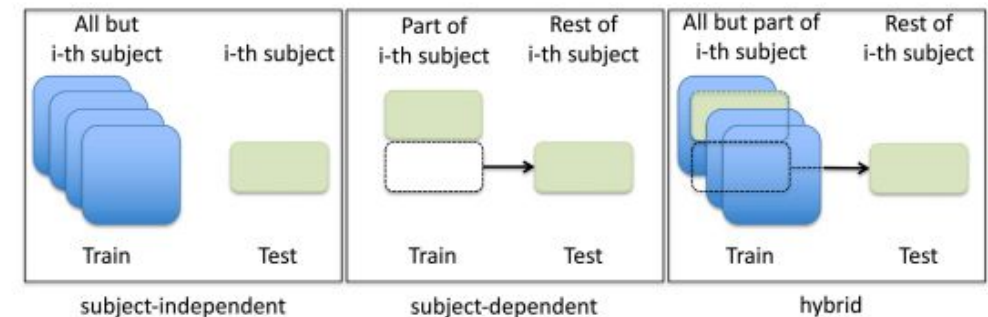
Università degli Studi di Milano

Copyright

These slides are from the teachers of this course. All the material is subject to copyright and cannot be redistributed without consent of the copyright holder. The same holds for audio and video-recordings of the classes of this course.

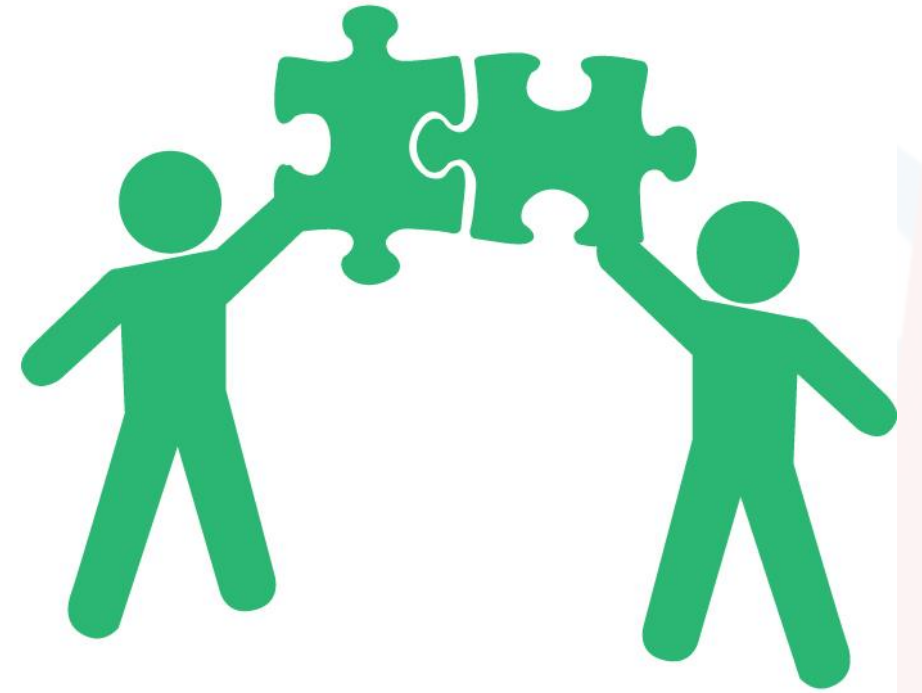
Recap on Training Models

- **Subject-independent models:**
 - trained on data from several subjects, with the objective of generalizing on unseen subjects
 - the most challenging setting!
- **Subject-dependent models:**
 - trained and tested on data from the same subjects
 - the most accurate!
- **Hybrid models:**
 - combine the strengths of the above two strategies
 - a bit less accurate than subject-dependent



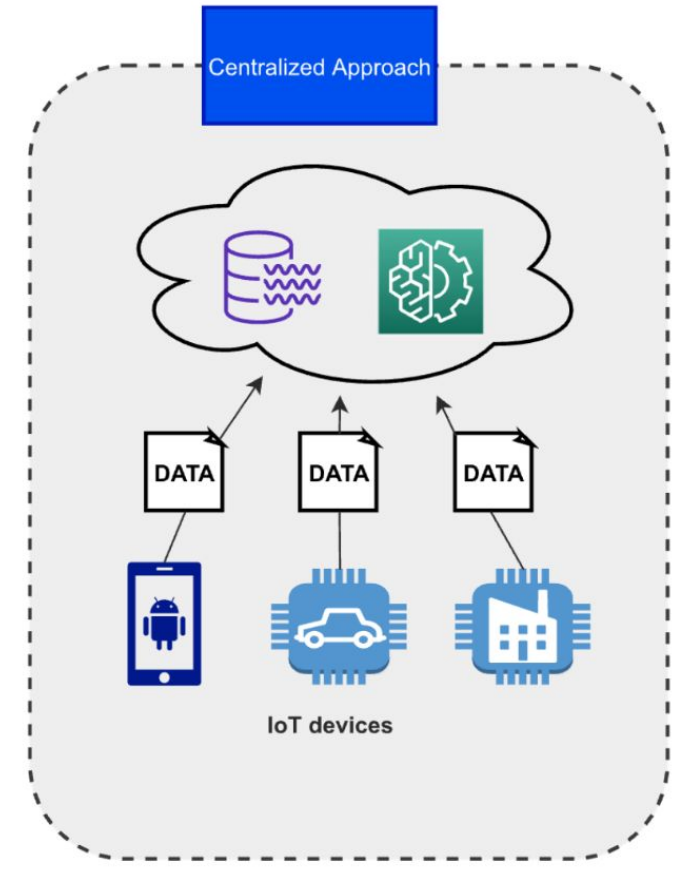
Collaborative Learning

- Ideally, the most accurate solution are based on subject-dependent models...
- ...but we know that the bottleneck is ***data acquisition and labeling***
- **Collaborative learning:**
 - distributing labeled data collection on a large number of users
 - build a single global model (subject-independent or hybrid, based on the scenario)



Centralized Models

- Each client transmits its available labeled data to a central server
- The server combines all the training data to generate a single training set
- A collaborative model is created using the large training set
- After training, the model can be used by:
 - the same clients that contribute to training
 - new clients



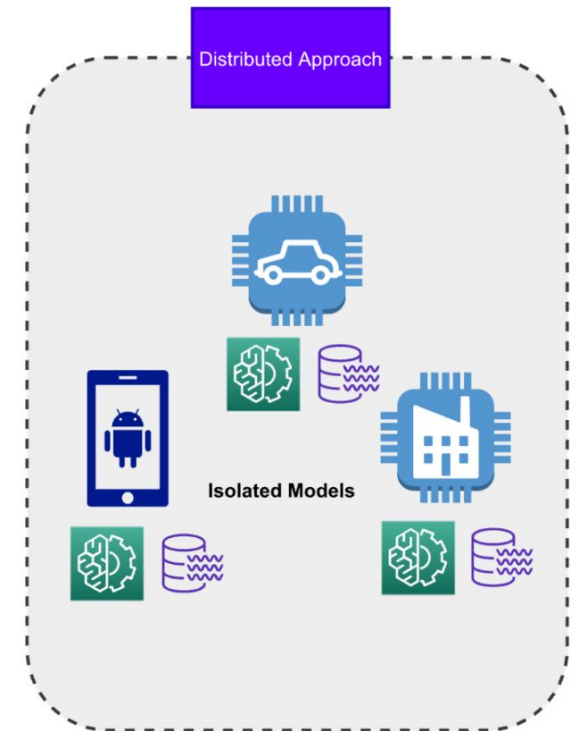
Problems of centralized approaches

- Building a single centralized model from a large number of users poses several challenges
- **Privacy:**
 - sensor data may reveal personal habits or health conditions
- **Scalability:**
 - communication latency
 - due to high amount of transmitted data
 - computational costs
 - due to training a model with large amounts of labeled data!



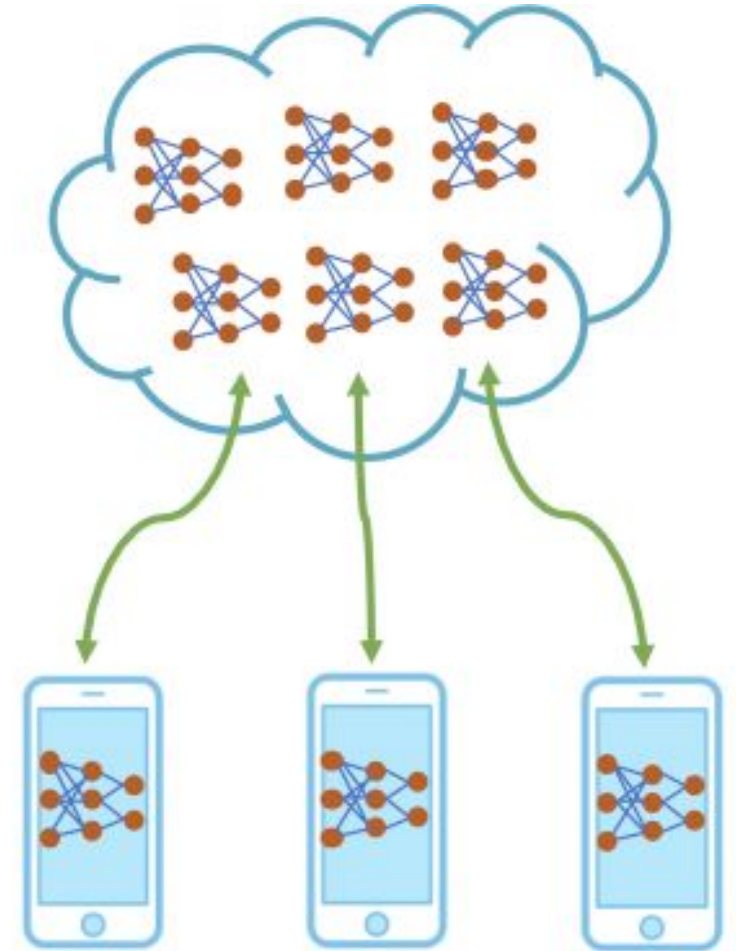
Distributed On-Site Learning

- Real-time systems may require distributed on-site ML
- Training and inference are only done on locally collected data
- The server distributes a pre-trained model and each device personalizes it
 - the privacy problem is mitigated!
- **Problems:**
 - nodes may have limited labeled samples
 - no benefits from peer's data
 - local models can not generalize

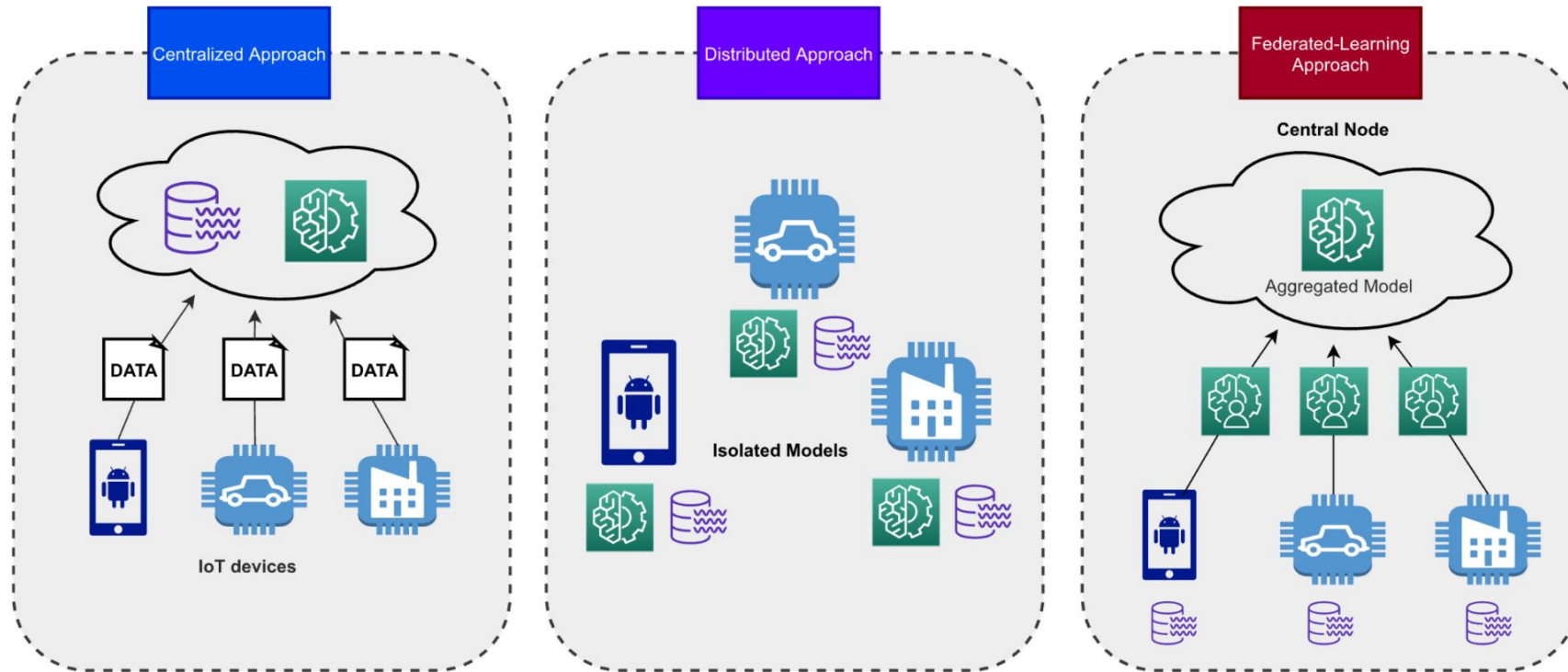


Federated Learning

- **Goal:** distributing training among the participating clients
 - each client uses its available labeled data to train a local model
 - only the local model parameters (and not data!) are shared with the server
 - the server aggregates them in a privacy-preserving fashion to generate a global model

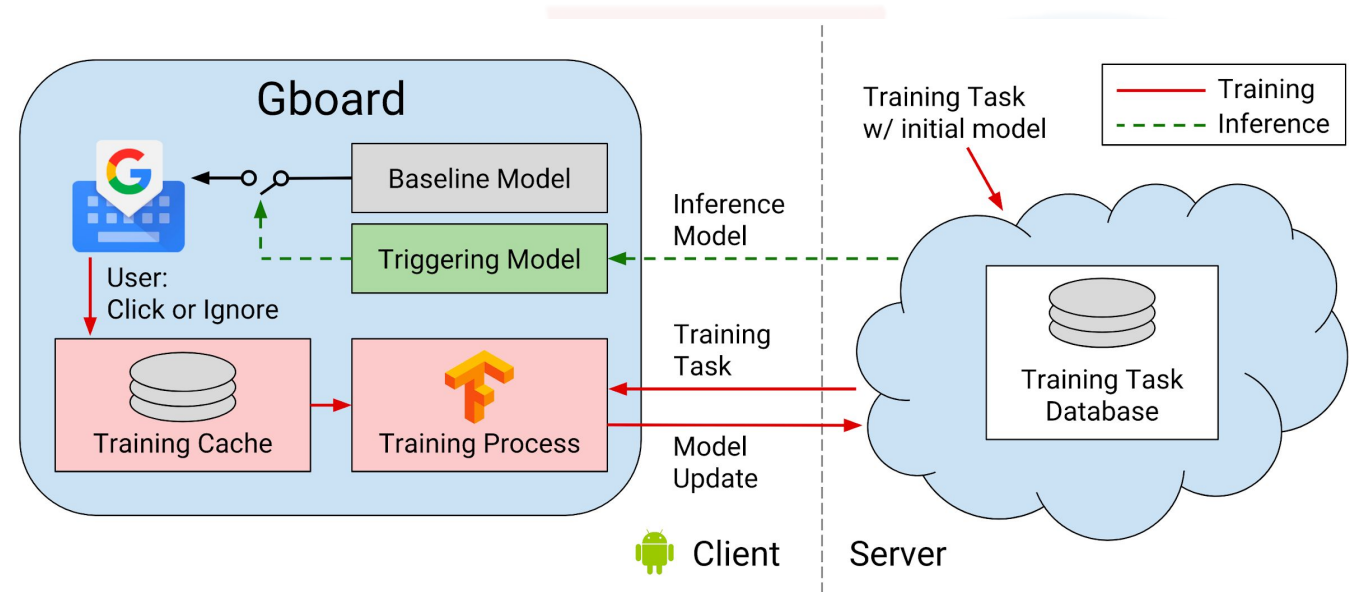


Centralized VS Distributed VS Federated



One of the first applications of Federated Learning: GBoard

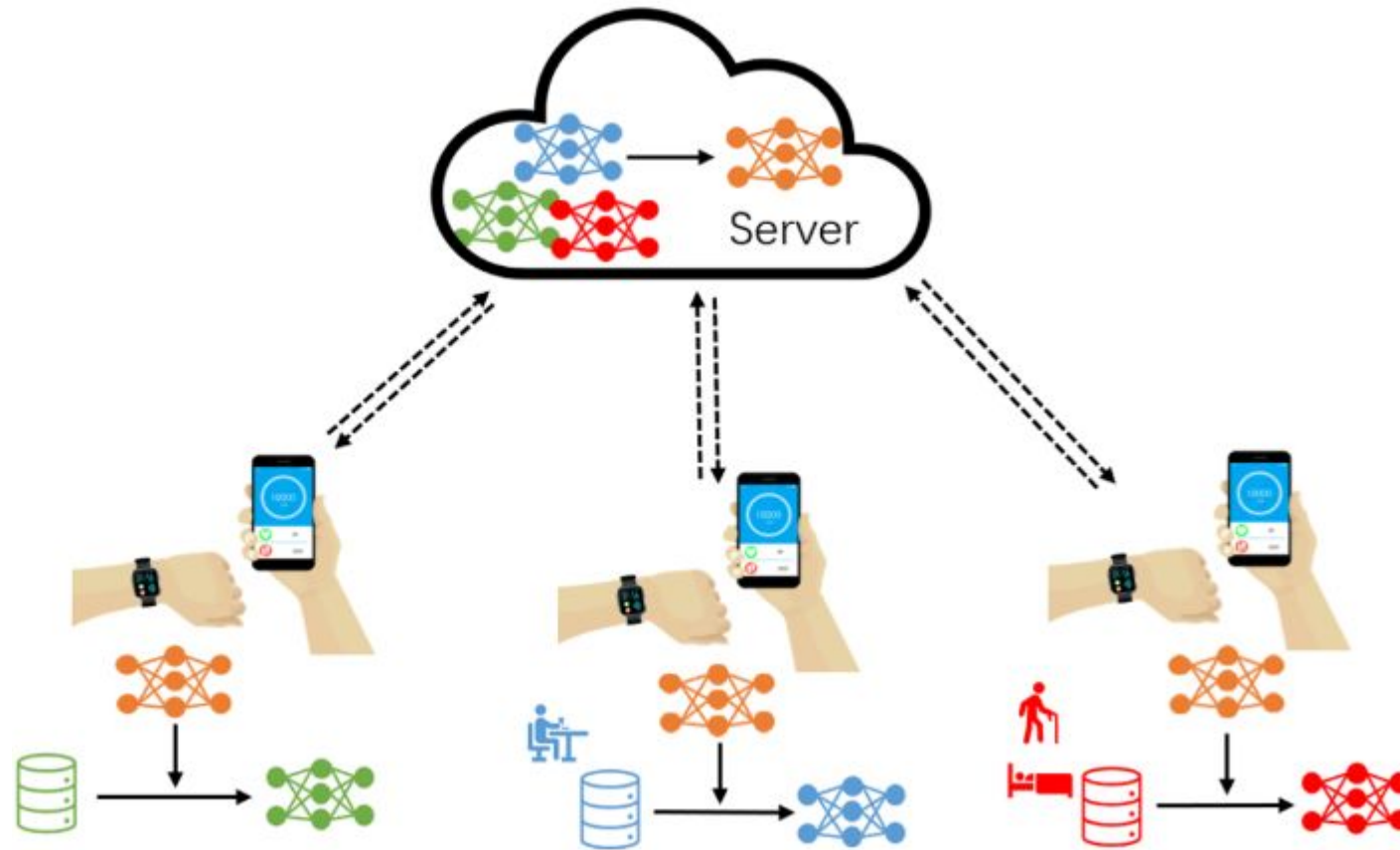
- Goal: suggesting the next word in Android devices keyboards to speed up writing
- When users “click” or “ignore” a suggestion, they implicitly provide labels
- A personal model is trained locally
- Local models are merged by a server to create a stronger model thanks to collaborative learning



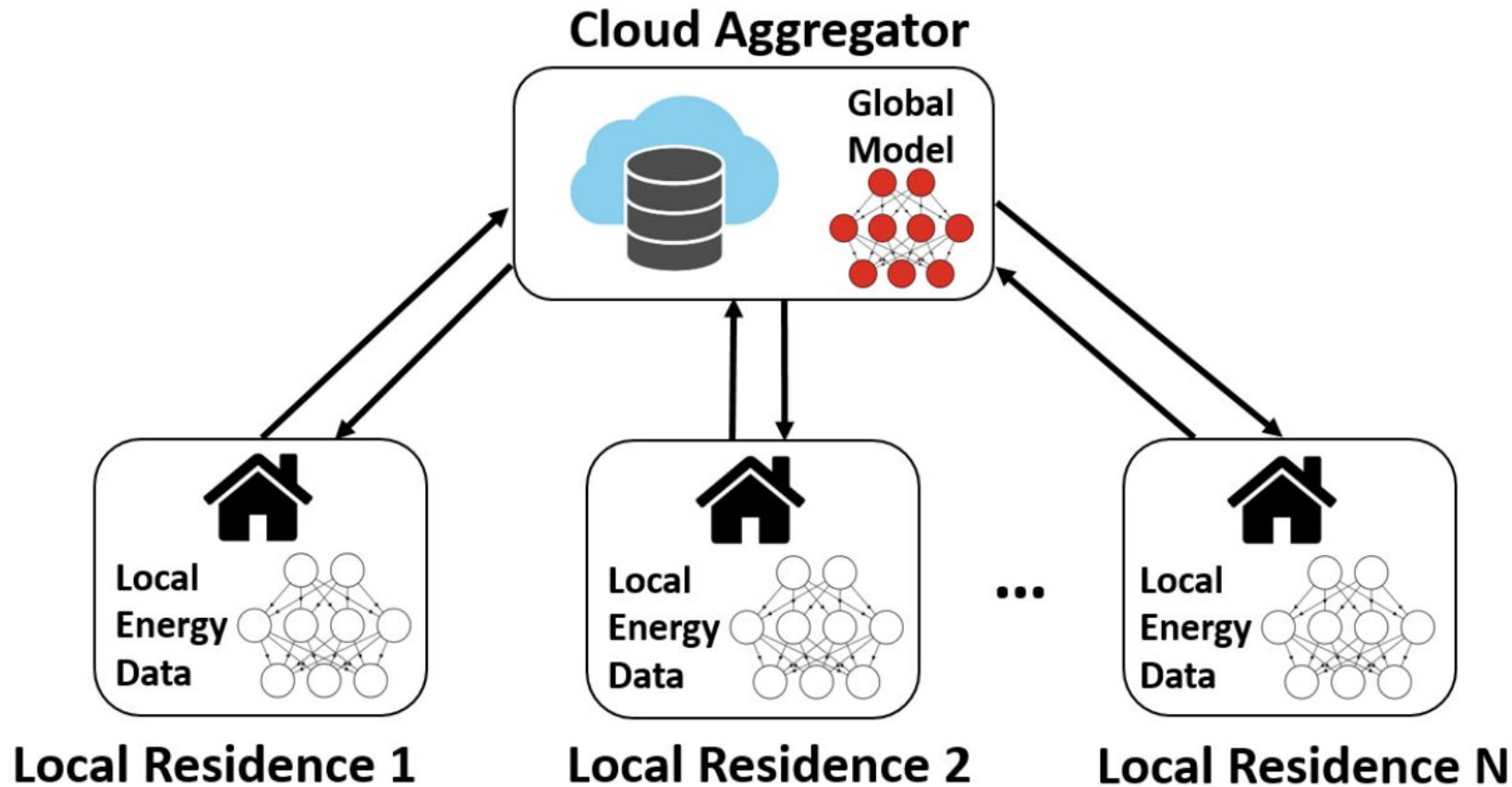
Yang, T., Andrew, G., Eichner, H., Sun, H., Li, W., Kong, N., ... & Beaufays, F. (2018). Applied federated learning: Improving google keyboard query suggestions. *arXiv preprint arXiv:1812.02903*.

Examples of applications in Ambient Intelligence

Federated Human Activity Recognition

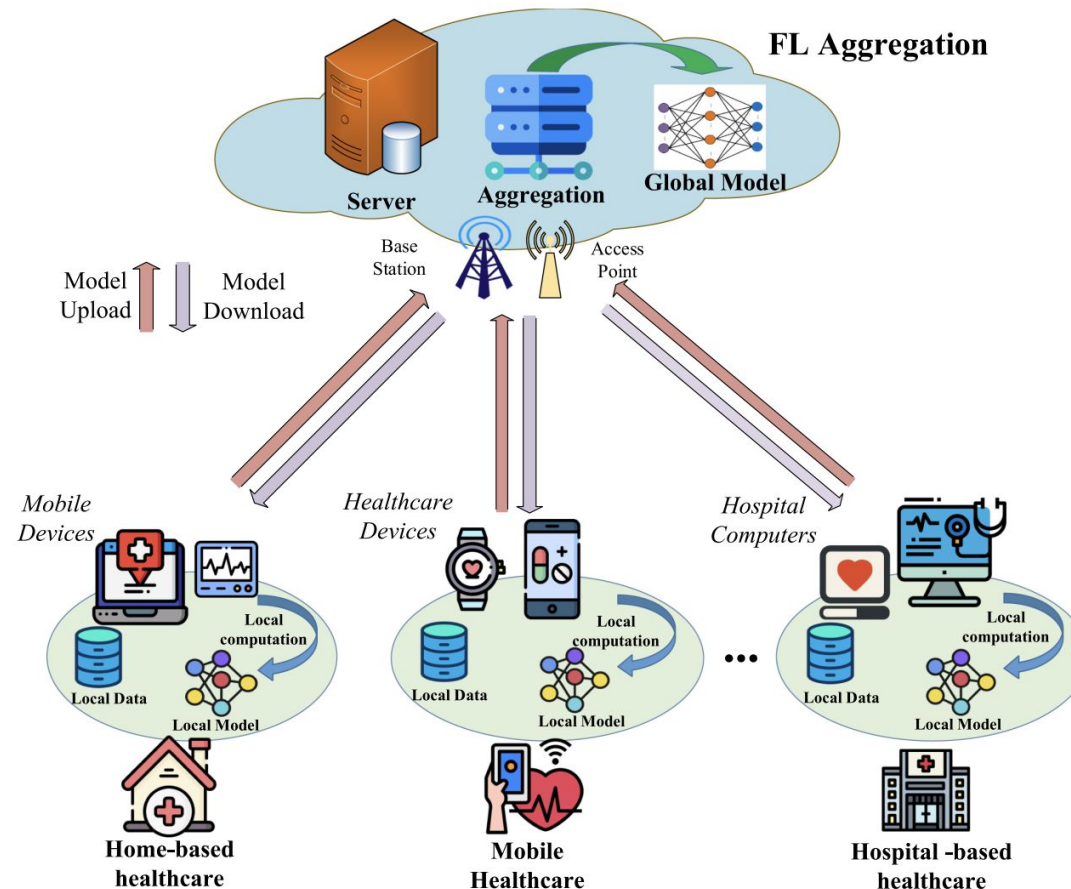


Federated Learning for Smart Energy

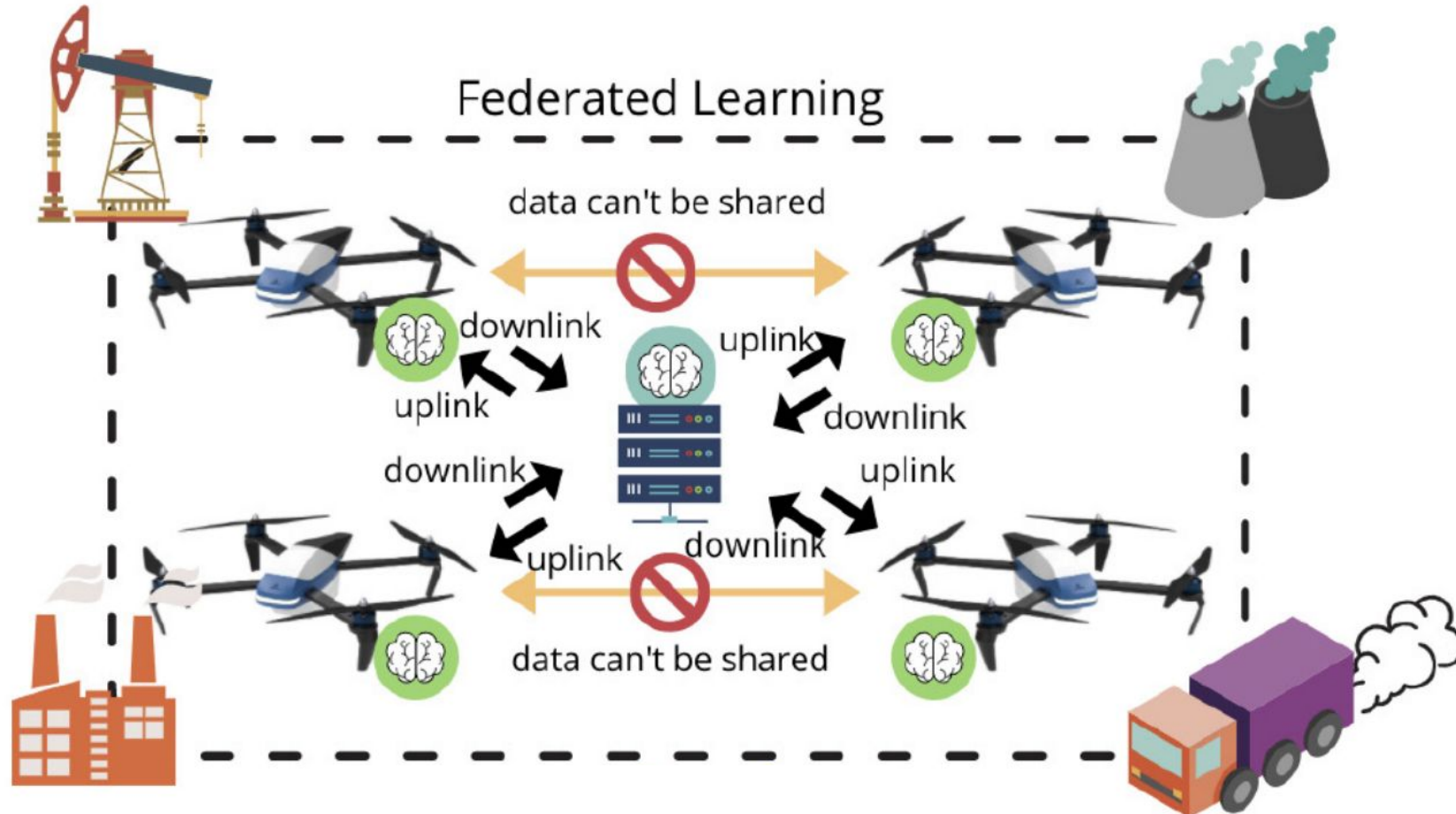


Federated Learning for Pervasive Healthcare

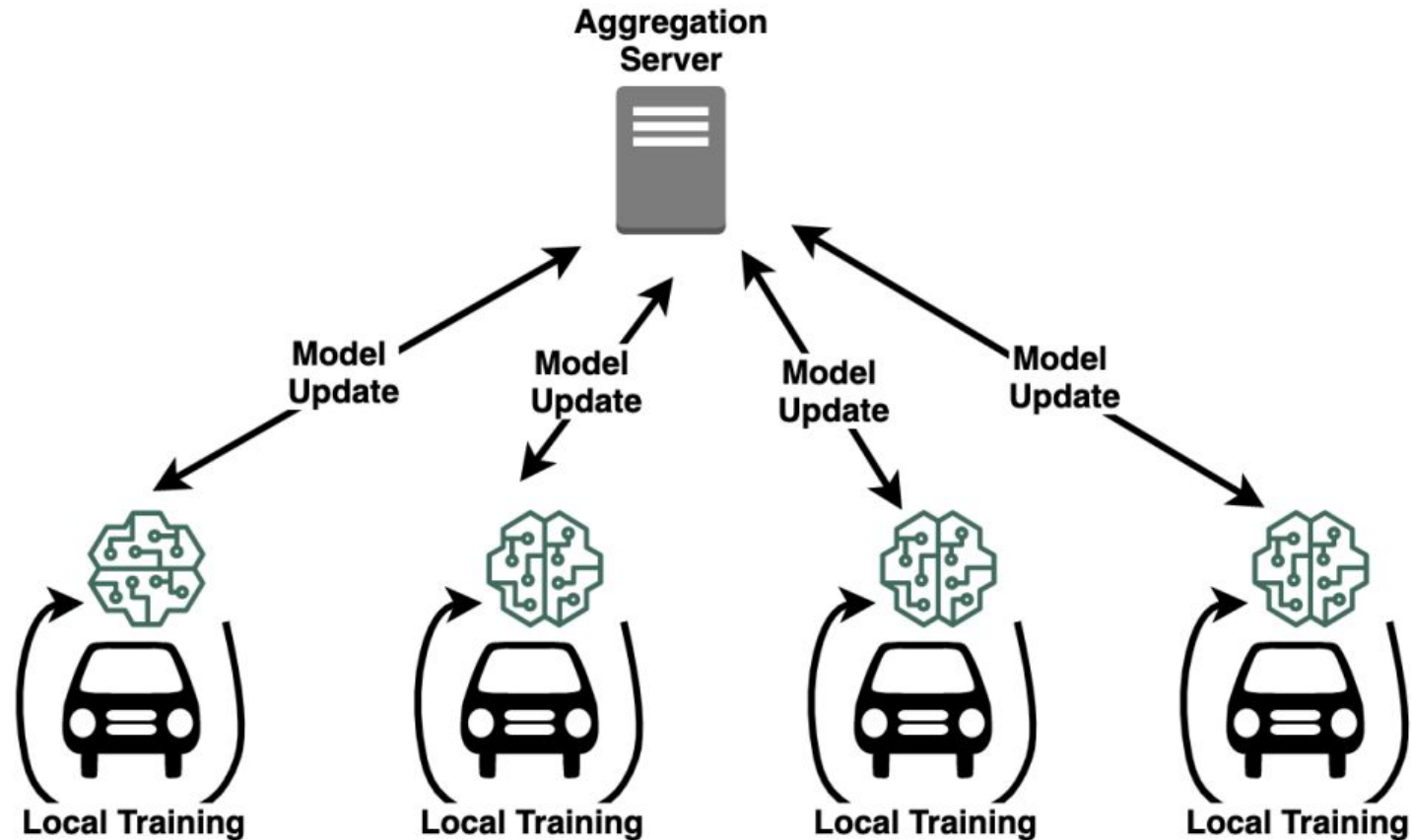
Nguyen, D. C., Pham, Q. V., Pathirana, P. N., Ding, M., Seneviratne, A., Lin, Z., ... & Hwang, W. J. (2022). Federated learning for smart healthcare: A survey. *ACM Computing Surveys (CSUR)*, 55(3), 1-37.



Federated Learning in Smart Cities (Pollution)



Federated Learning for Autonomous Driving

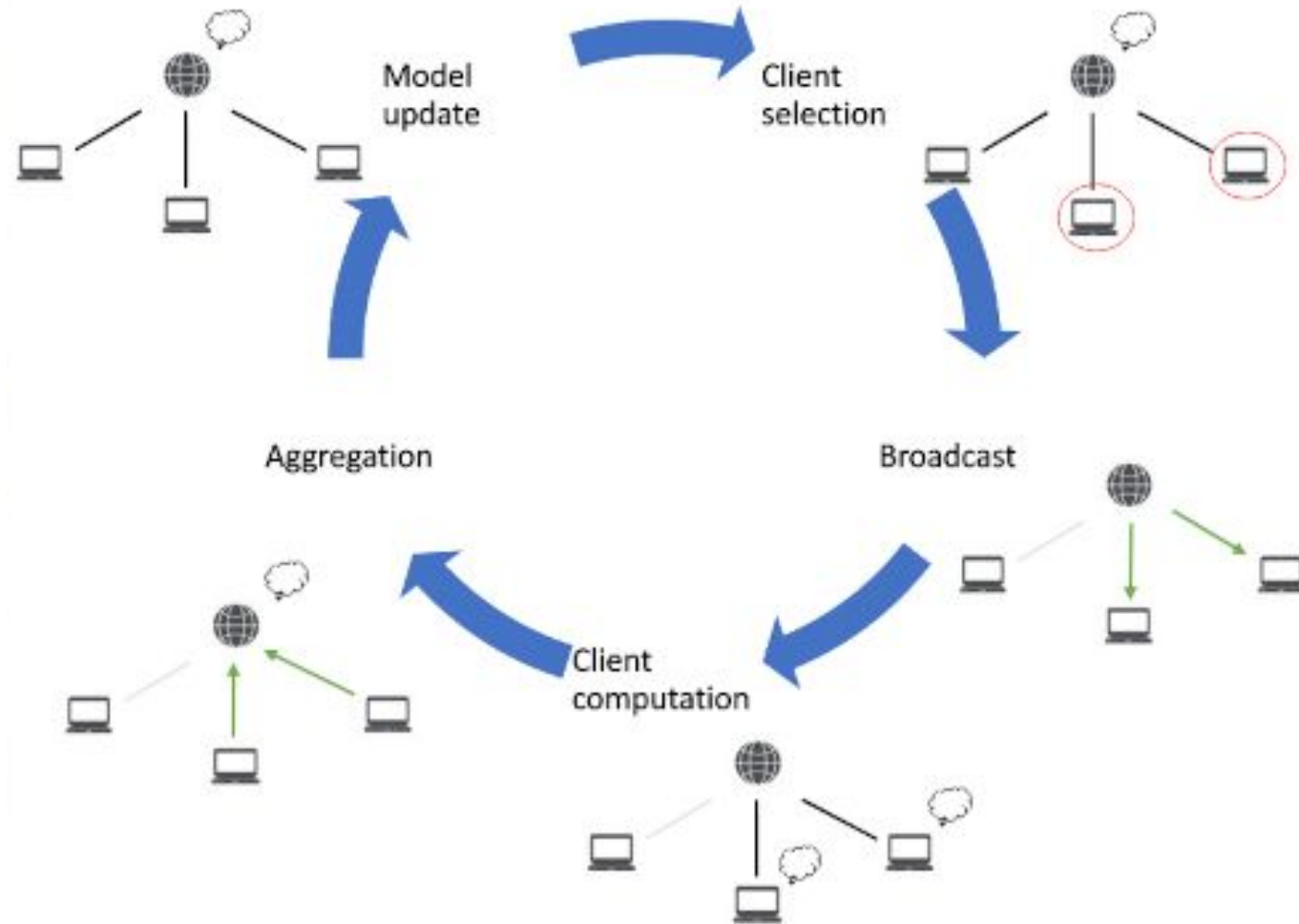


The basic Federated Learning Approach

Federated Learning mechanism

- Periodically, the server requires to update the ***global model***
- It selects a certain number of available clients
 - e.g., randomly or using some strategies
- Each selected client
 - receives the global model
 - uses available labeled data to train it
 - sends the resulting local model parameters to the server
- Once the server receives the updates from selected clients, it aggregates them
 - the result is an updated version of the global model!
- For each global model update, this process is repeated until convergence
 - each iteration is called ***communication round***

A communication round



A naive approach: FedSGD

- Each client locally computes the average gradient on its local data at current model

$$F_k(w) = \frac{1}{n_k} \sum_{i \in \mathcal{P}_k} f_i(w)$$

$$g_k = \nabla F_k(w_t)$$

- The server:
 - receives the local gradients from the clients
 - performs a weighted aggregation of gradients
 - updates the model with one step of SGD

$$w_{t+1} \leftarrow w_t - \eta \sum_{k=1}^K \frac{n_k}{n} g_k$$

FedAVG: Federated Averaging

- Each client locally performs several numbers of SGD steps:

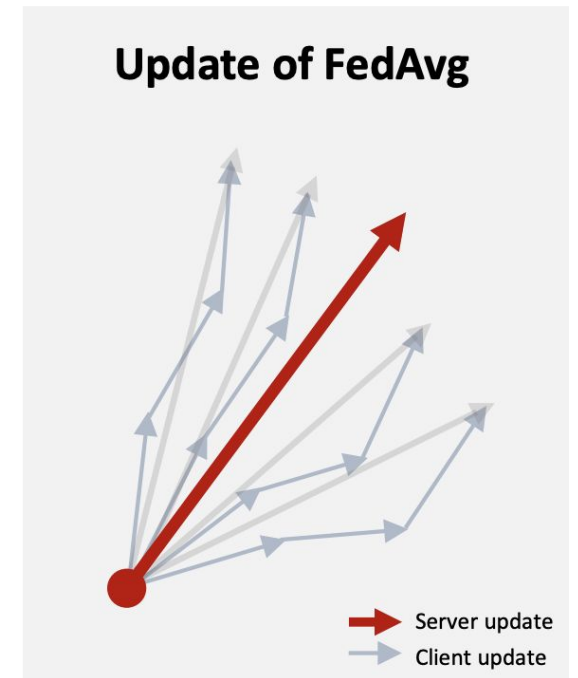
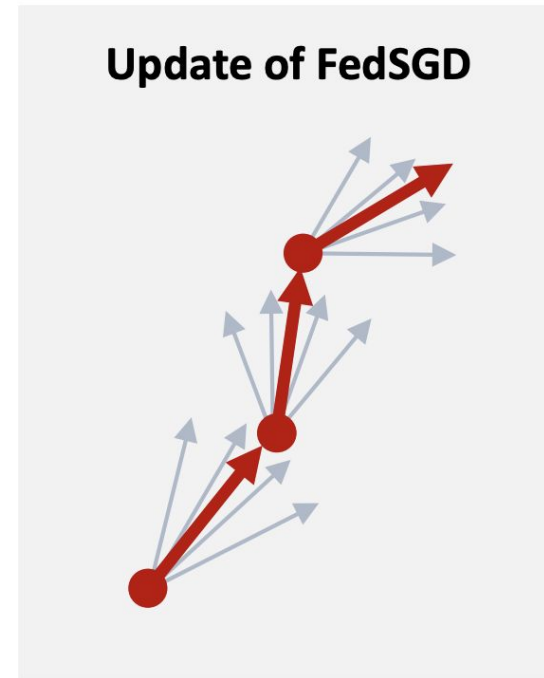
$$w_{t+1}^k \leftarrow w_t - \eta g_k$$

- The server:
 - receives the **weights** of the local model
 - performs a weighted aggregation of the weights
 - the result is the new global model model

$$w_{t+1} \leftarrow \sum_{k=1}^K \frac{n_k}{n} w_{t+1}^k$$

FedSGD vs FedAVG

- FedAVG requires more computation on each client (i.e., running several steps of SGD)
- Nonetheless, it allows the model converge with a significantly reduced number of communication rounds!
- FedAVG is the standard approach generally used in FL



Client Selection

- Federated Learning is designed for scenarios where a large number of clients participate (e.g., thousands)
- Considering all the clients to update the global model is not scalable
 - also some of them may not be available
- FL client selection is the process that determines which client devices are chosen in each training round
- ***A simple strategy:*** random sampling from the *available* devices
 - since local training is costly in terms of energy consumption and may impact the device usability, a device may be *available* (for instance) in the night while charging
 - the server asks for availability and samples a random number of available clients

Client Selection through Utility

- Instead of random sampling, each client can be measured for its utility
- The clients with the best utility measurements are selected for model training and aggregation
- Several methods:
 - **Statistical Utility:** the usefulness of a client's local update to the global model
 - e.g., the number of data samples used to train the local model, the local cumulative loss, the difference between local and global model
 - **System Utility:** due to different hardware configurations, FL clients may lead to different overheads (e.g., training and transmission time)
 - indeed, slow devices (i.e., stragglers) can deteriorate the overall training performance by prolonging the training round
 - in this case, a possibility is considering a threshold on the response time!

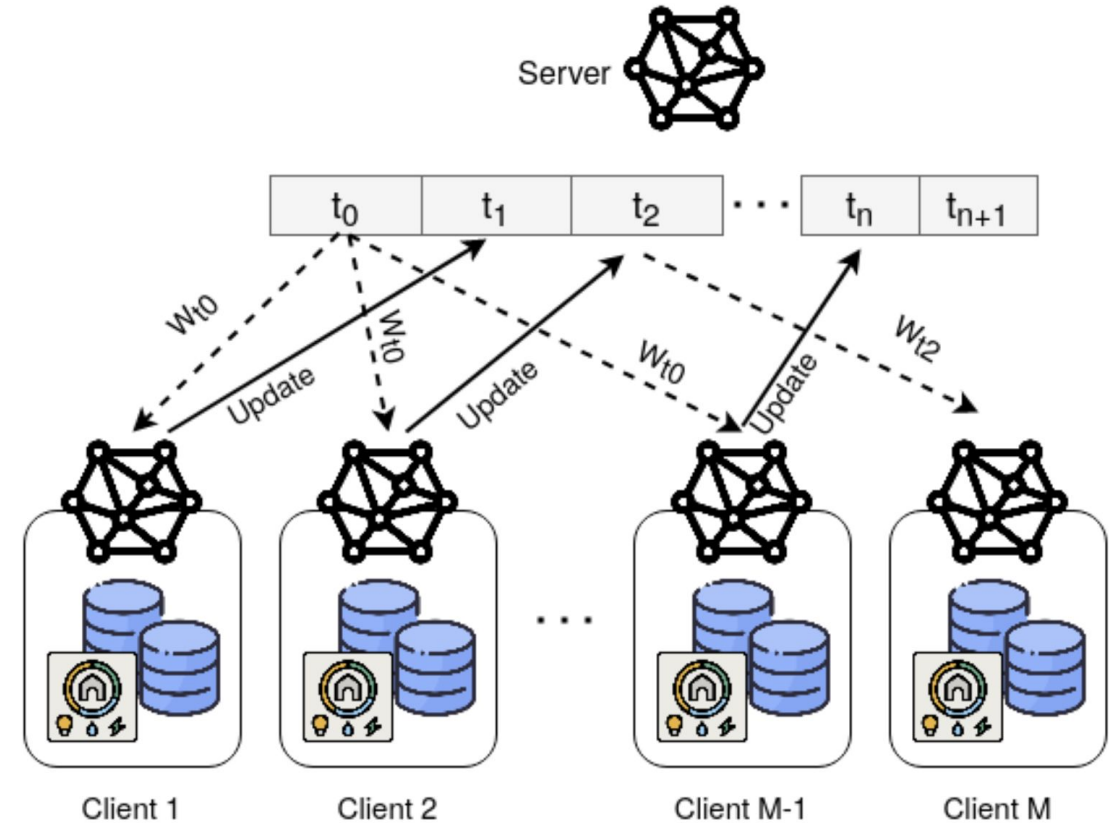
Asynchronous Federated Learning

Problems of Synchronous FL

- Classic FL methods are based on synchronous update:
 - a subset of clients is selected, and aggregation is performed only when every client transmits its local model updated
- This may be not always optimal when the model has to be updated frequently and quickly:
 - some devices may become unresponsive (e.g., failures, network problems)
 - different devices may have different computational resources, hence the time required for local training may significantly differ
 - devices with significant delays are called ***straggler devices***
- When there is the need of updating the global model frequently to keep it constantly updated, ***asynchronous FL*** strategies may be considered
- The goal is reaching convergence quicker than FedAVG!

Asynchronous FL

- Each client transmits an update of the local model as soon as it is available
- The server keeps track of the timestamps at which each global model is updated to weight model updates
 - intuitively, if a local model arrives “late” it may contain old information and its contribution to the global model should impact less



A solution: FedAsync

Xie, C., Koyejo, S., & Gupta, I. (2019). Asynchronous federated optimization. *arXiv preprint arXiv:1903.03934*.

- The server periodically schedules a global model update
- It contacts each client separately, by transmitting the current model x_τ
 - τ is the timestamp at which the global model is created
- As soon as possible, the client submits (x_{new}, τ) to the server
 - x_{new} is the locally trained model, τ is the timestamp of the global model on which the client generated x_{new} with local training
- The server immediately updates the current model with the following formula:

$$x_t \leftarrow (1 - \alpha_t)x_{t-1} + \alpha_t x_{new}$$

Staleness

- In order to update the global model, the server assesses the *staleness* of the local model update
 - by analyzing the difference $t - \tau$
 - intuitively, larger staleness results in greater error when updating the global model (i.e., using old information)
- Hence, α_t controls the update of the global model considering the staleness:

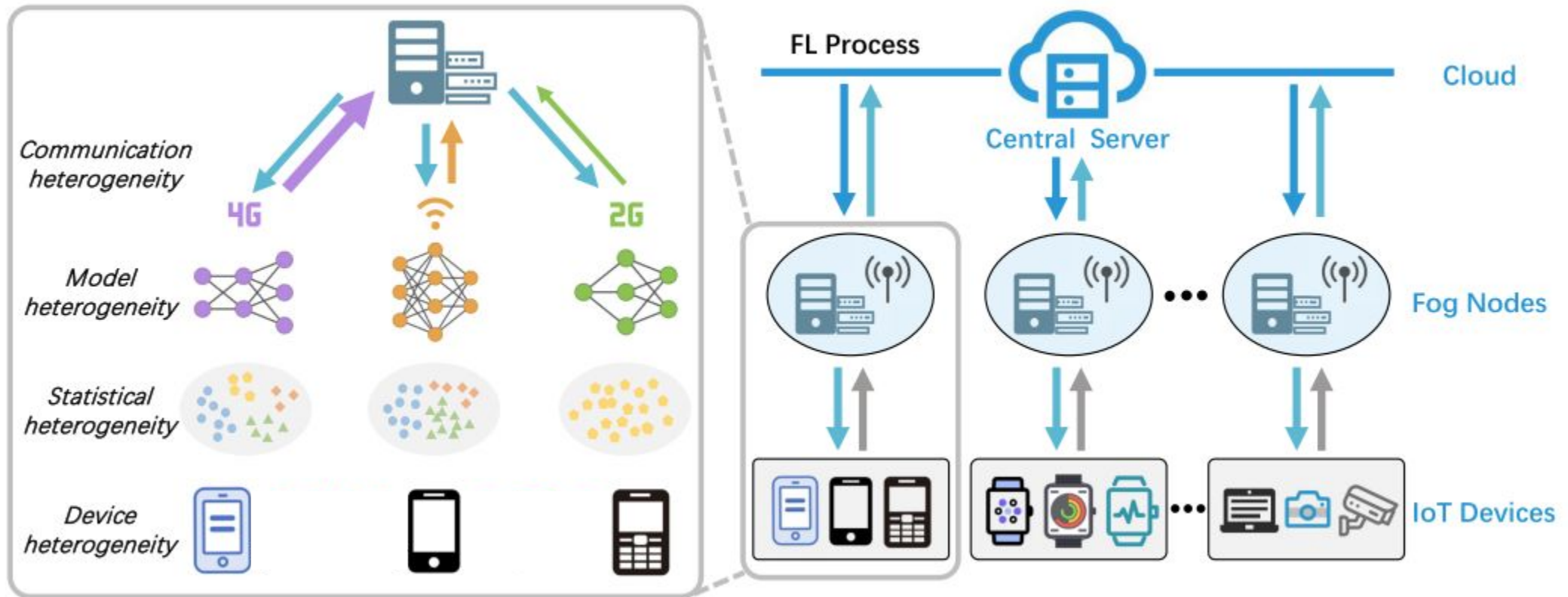
$$\alpha_t \leftarrow \alpha \times s(t - \tau)$$

- A possible formula for staleness is the following:

$$s_a(t - \tau) = \frac{1}{t - \tau + 1}$$

Heterogeneity in FL

Clients may be heterogeneous!

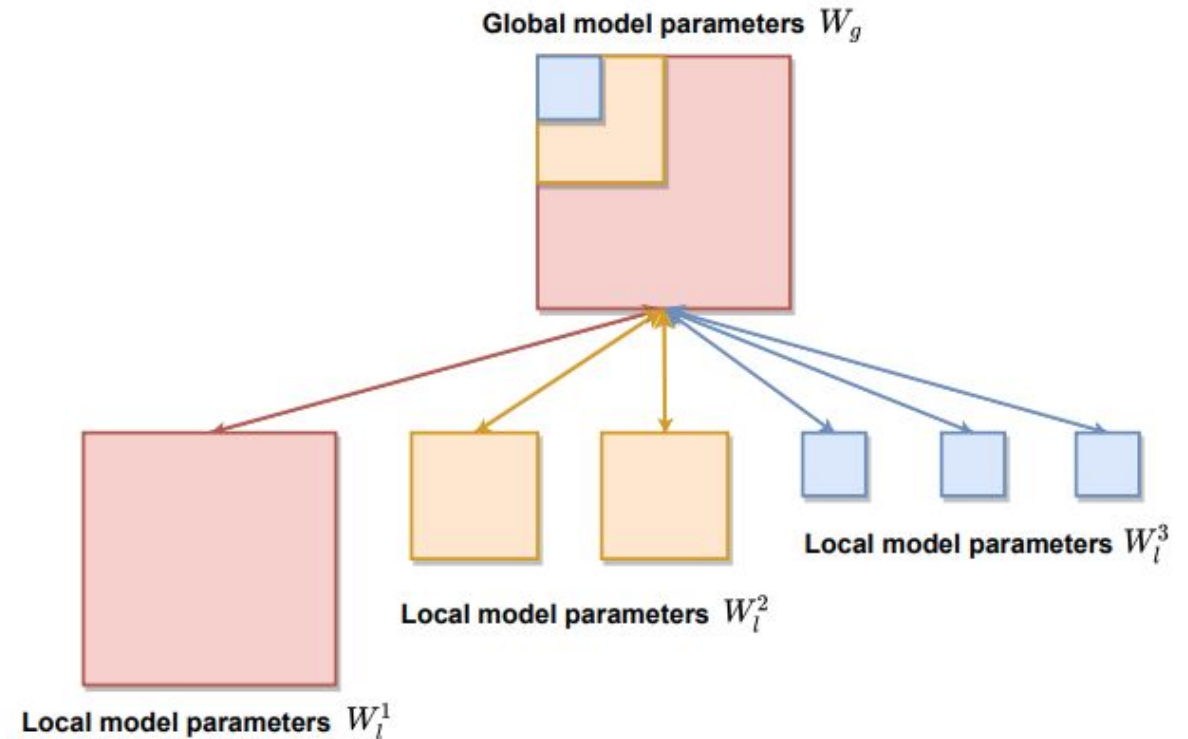


Model Heterogeneity

- A widely accepted assumption is that local models share the same architecture as the global model
- However, this would require to limit the global model complexity for resource constrained devices
- Indeed, the computation and communication capabilities of each client may vary significantly (even dynamically!)

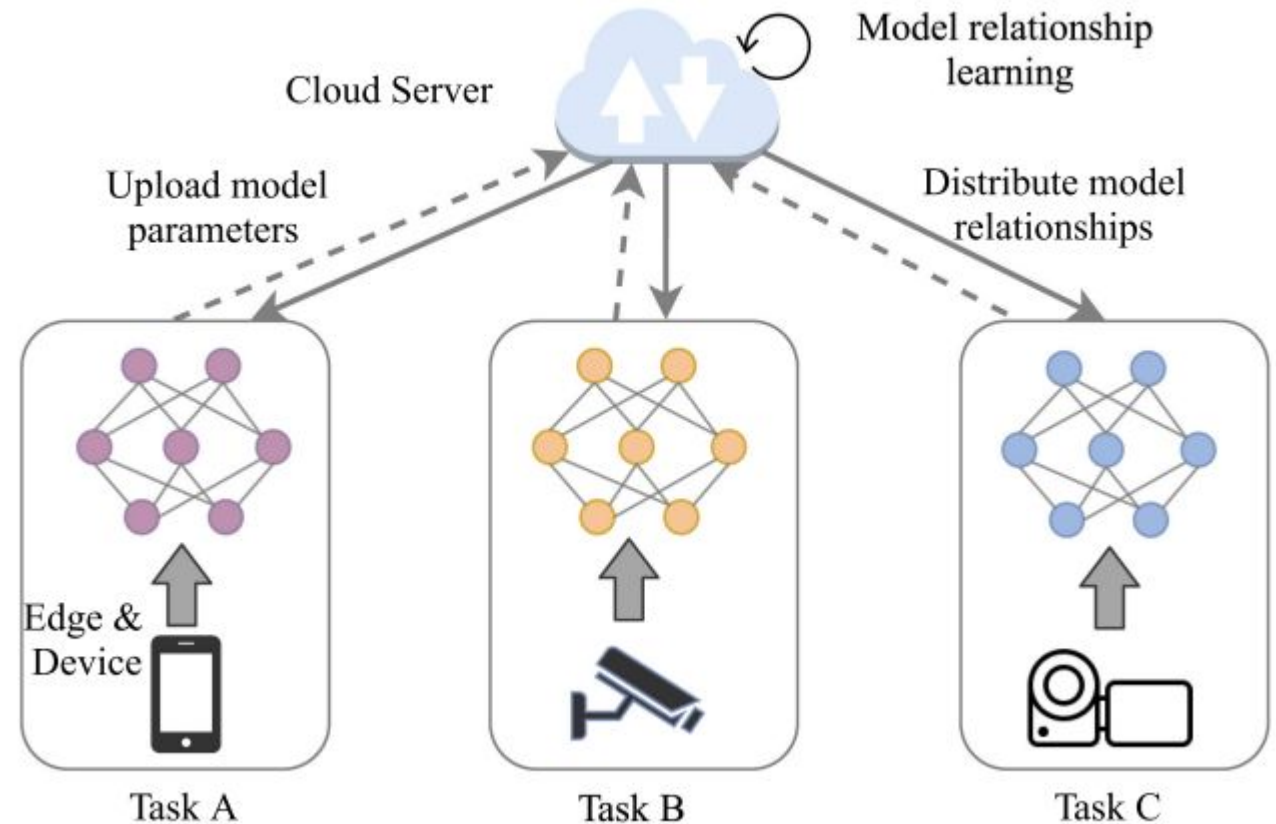
A possible solution: HeteroFL

- A solution to this problem is to provide “smaller” versions of the global model to the resource constrained devices
- Idea:
 - the computation complexity of the global model is reduced by reducing the **width** of hidden channels.
 - consequently reducing the number of parameters!
 - Local and global model architectures are within the same model class to stabilize global model aggregation
 - Each client contributes to a different number of parameters based on model size!



Task Heterogeneity: An Approach based on Multi-Task Learning

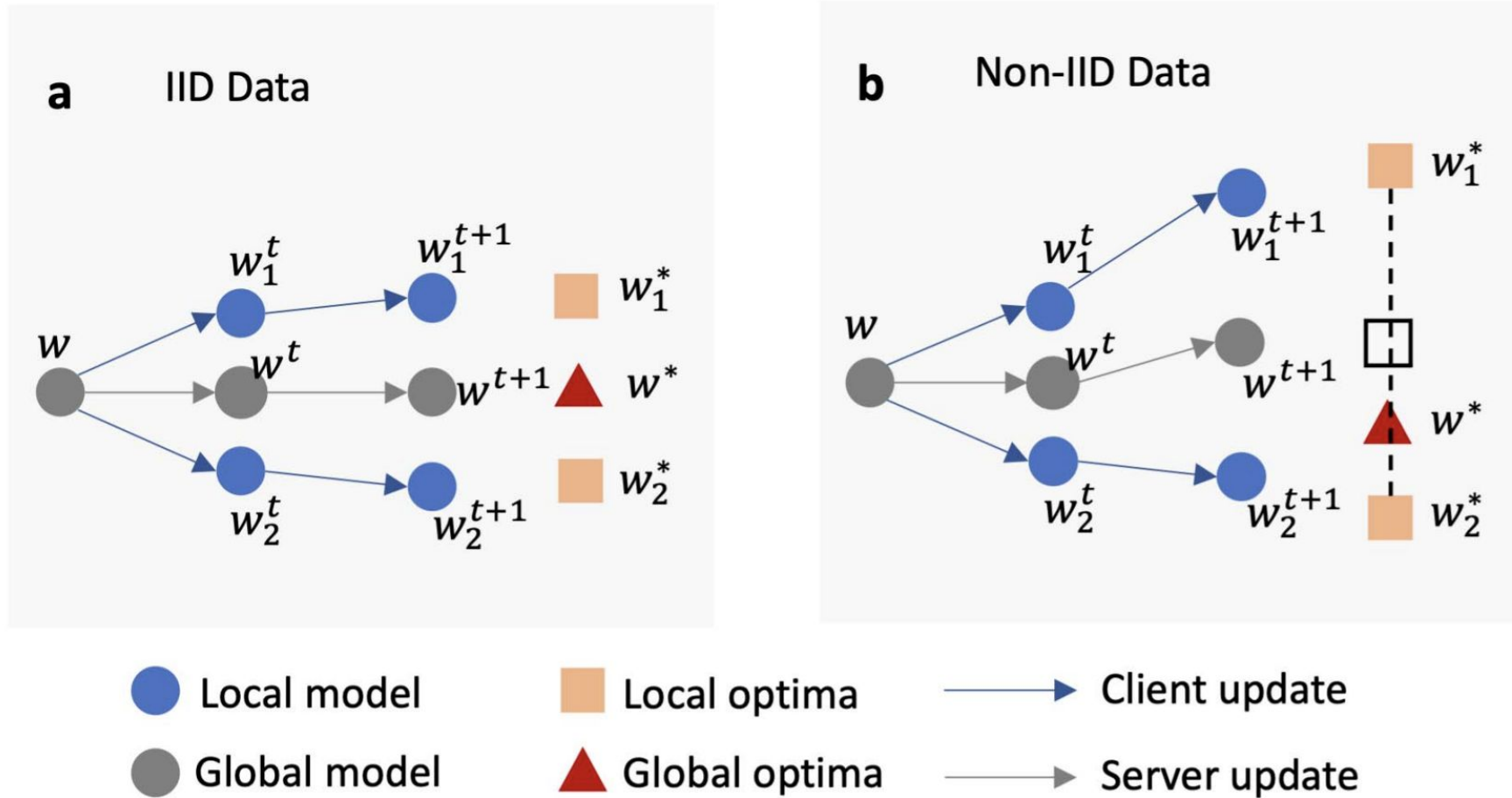
- By keeping part of the parameters private and part of the parameters public, it is possible to rely on **transfer learning** to collaboratively learn multiple tasks
- FL makes it possible to collaboratively learn the **shared layers** that may be general for several tasks
- Each client may fine-tune the local model on a specific task



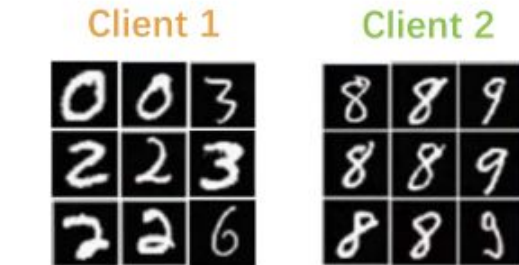
Statistical Heterogeneity: the non-IID problem

- The global model should generalize over a large number of clients
- The local model on each client typically refers to a specific user
 - e.g., specific physical characteristics, age, habits
- Data from different clients are non identically and independently distributed (non-IID)!
- FL solutions that are only based on local optimization (e.g., FedAVG) are not adequate

IID vs Non-IID



Types of Skewness in non-IID clients

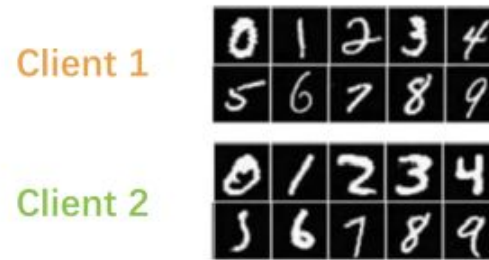


Label distribution skew

	image	label
Client 1		<i>Fear</i>
Client 2		<i>Surprise</i>

Label preference skew

(a) Label Skew

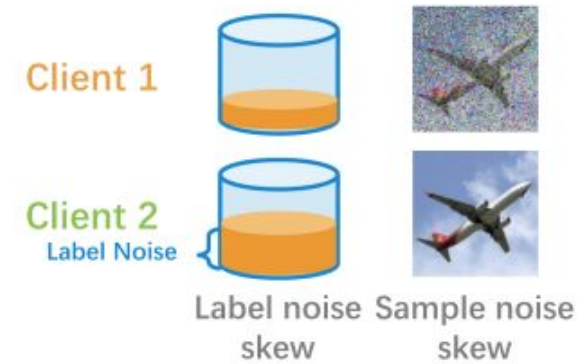


Feature distribution skew

	image	label
Client 1		<i>Dog</i>
Client 2		<i>Dog</i>

Feature condition skew

(b) Feature Skew



(c) Quality Skew



Quantity Skew

(d) Quantity Skew

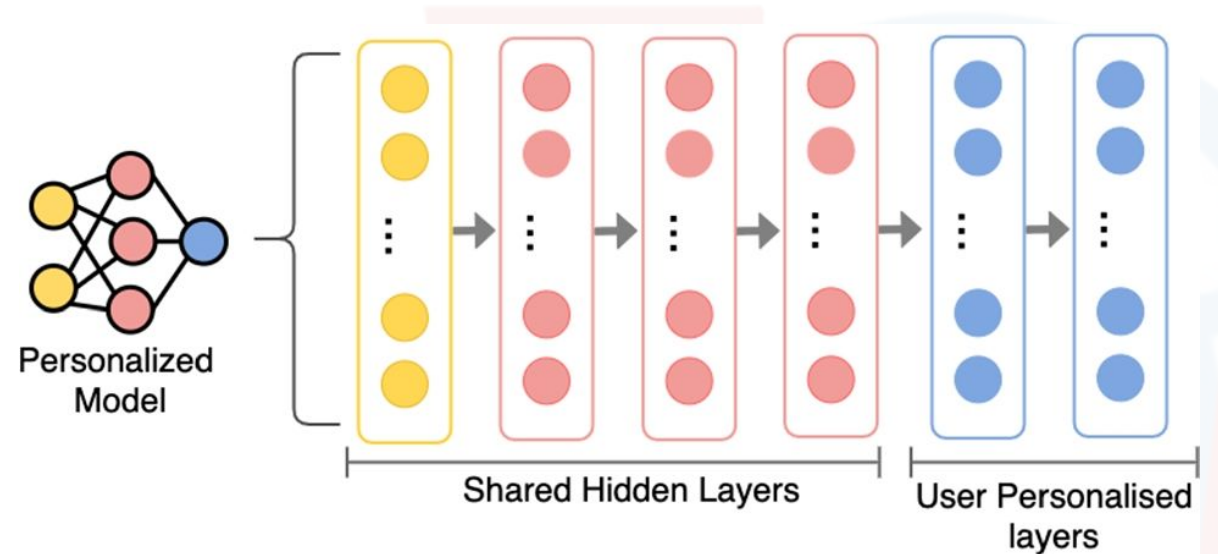
Skewness in sensor data time series

Two or more datasets are non-IID if some *skewness* (asymmetry in data distributions) can be observed. Most common skewnesses in sensor data from different subjects:

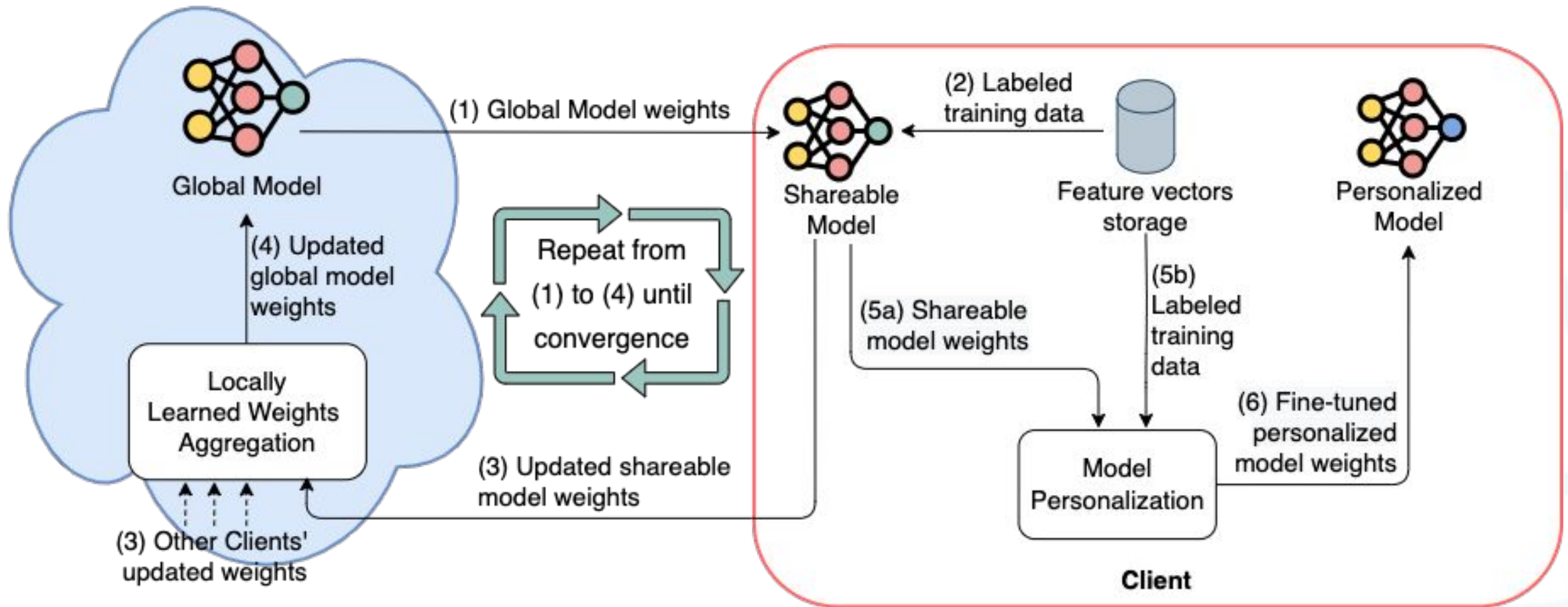
- ***Feature distribution skew***
 - each subject may generate peculiar sensor data (e.g., *walking* inertial sensor patterns generated by a young subject vs the ones generated an elder subject)
- ***Label distribution skew***
 - different subjects may have different routines (e.g., a sporty subject spends more time *running* than a sedentary one)
- ***Quantity distribution skew***
 - different subjects may have significantly different availability of labeled data

Tackling non-IID using transfer learning

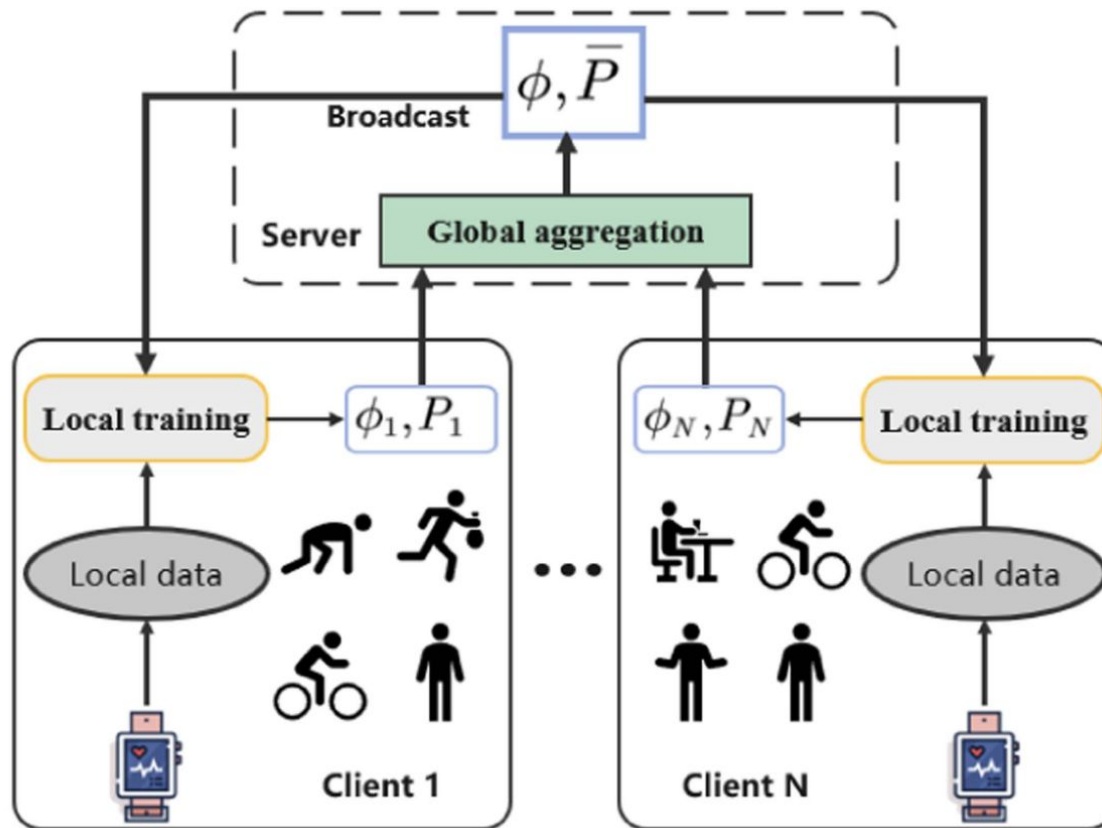
- Idea:
 - the layers closest to input represent features that may be general for several clients (e.g., feature extraction component)
 - the layers closest to the output represent specific characteristic of a client (e.g., fully connected output layers)
- After the server generates a global model, each client can locally ***fine-tune*** the personalized layers!



Personalization using fine-tuning



ProtoHAR: using class prototypes to mitigate label and quantity skewness



- **Motivation:** label and quantity skewnesses
- Each client transmits to the server:
 - the parameters of the local feature extractor
 - a prototype (in the latent space) of each activity
- The server creates a global feature extractor and global class prototypes
- During local training, each client should minimize the distance of local prototypes with global prototypes
 - so that it can perform better on locally under-represented classes!

ProtoHAR: the loss function

$$F_{\phi} = \min_{\phi_i} L_i(\phi_i, h_i) + \lambda L_R(P_i^{(j)}, \bar{P}_i^{(j)})$$

standard classification loss

distance between local prototypes and global prototypes

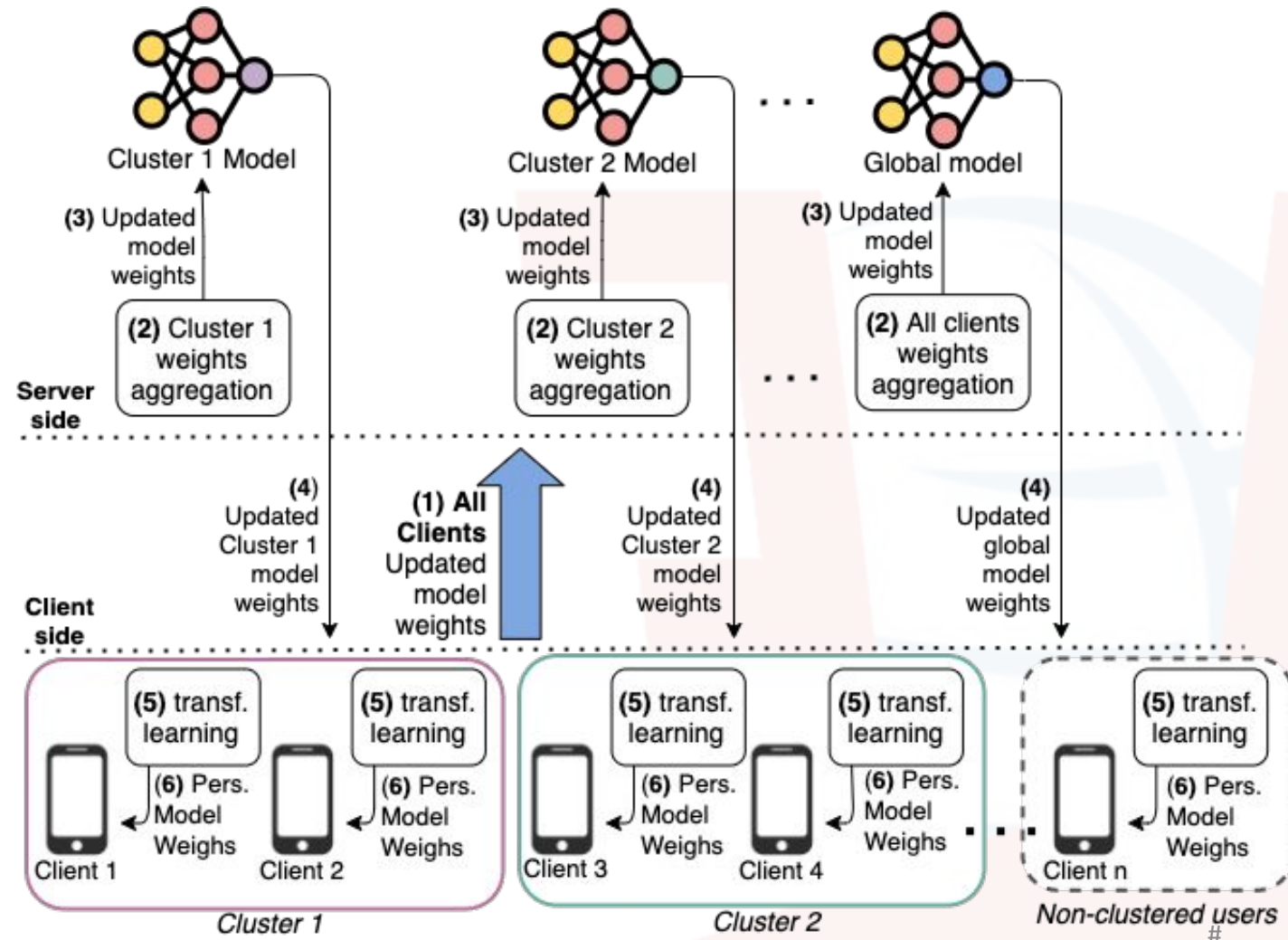
Federated Clustering

From single to multiple global models

- The previously proposed solutions to mitigate non-IID are based on a **single global model** fine-tuned on each client
- However, it is challenging to determine a good trade-off between personalization and generalization
 - each local model should be personalized
 - the global model should generalize over unseen clients
- In the following, we will talk about Federated Clustering, a method tackling the non-IID problem by grouping clients by *similarity*
- This method leads to a global model for each group
 - the global model of each group will likely consider IID clients, since they are similar between them!

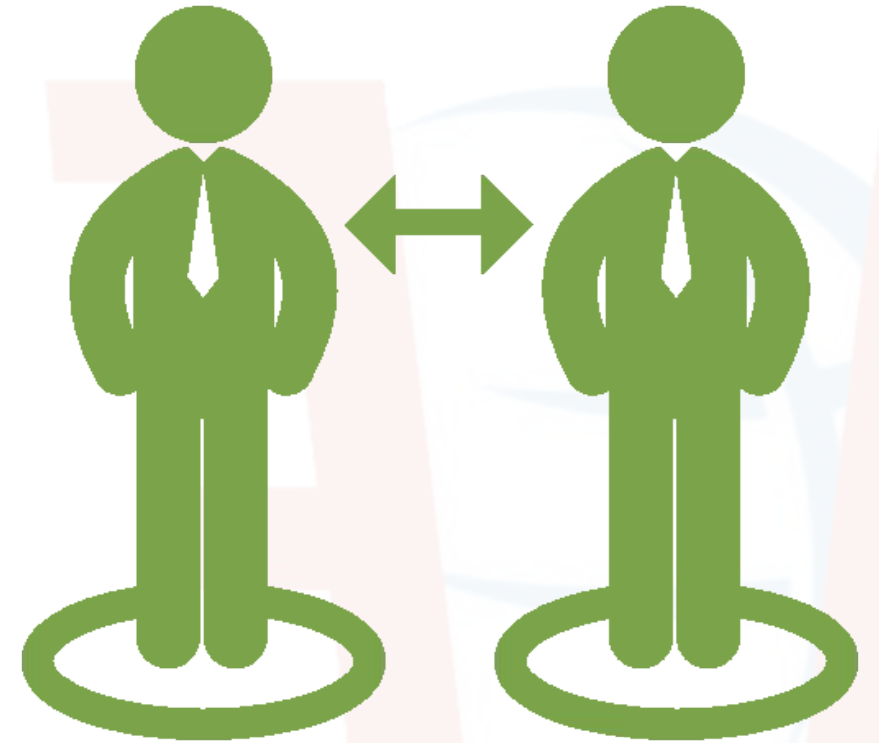
Federated Clustering

- A global model for each cluster (i.e., group of similar clients)
- Each client transmits updated local weights only to its cluster model
- Personalization using transfer learning is performed locally
- A general global model trained with data from all the clients is maintained for generalization and for *non-clustered clients*



Similarity between users in FL

- Users can be considered similar if they share similar sensor data patterns
- However, raw data is not available in FL settings and it is not possible to directly compute similarity
- **Idea:**
 - it is very likely that two local models with similar weights have also been trained with similar sensor data patterns
 - consequently, it is also likely that the corresponding users are similar



Similarity (cont.)

- Usually, the *cosine similarity* between model parameters is computed
- However, only the *personal layers* should be considered, since they reflect the most specific characteristics of each client!
 - the choice of which layers should be considered *personal* strictly depends on the underlying model

$$\text{sim}(\mathbf{w}_i, \mathbf{w}_j) = \frac{\text{pers}(\mathbf{w}_i) \cdot \text{pers}(\mathbf{w}_j)}{\|\text{pers}(\mathbf{w}_i)\| \|\text{pers}(\mathbf{w}_j)\|}$$

Hierarchical Clustering

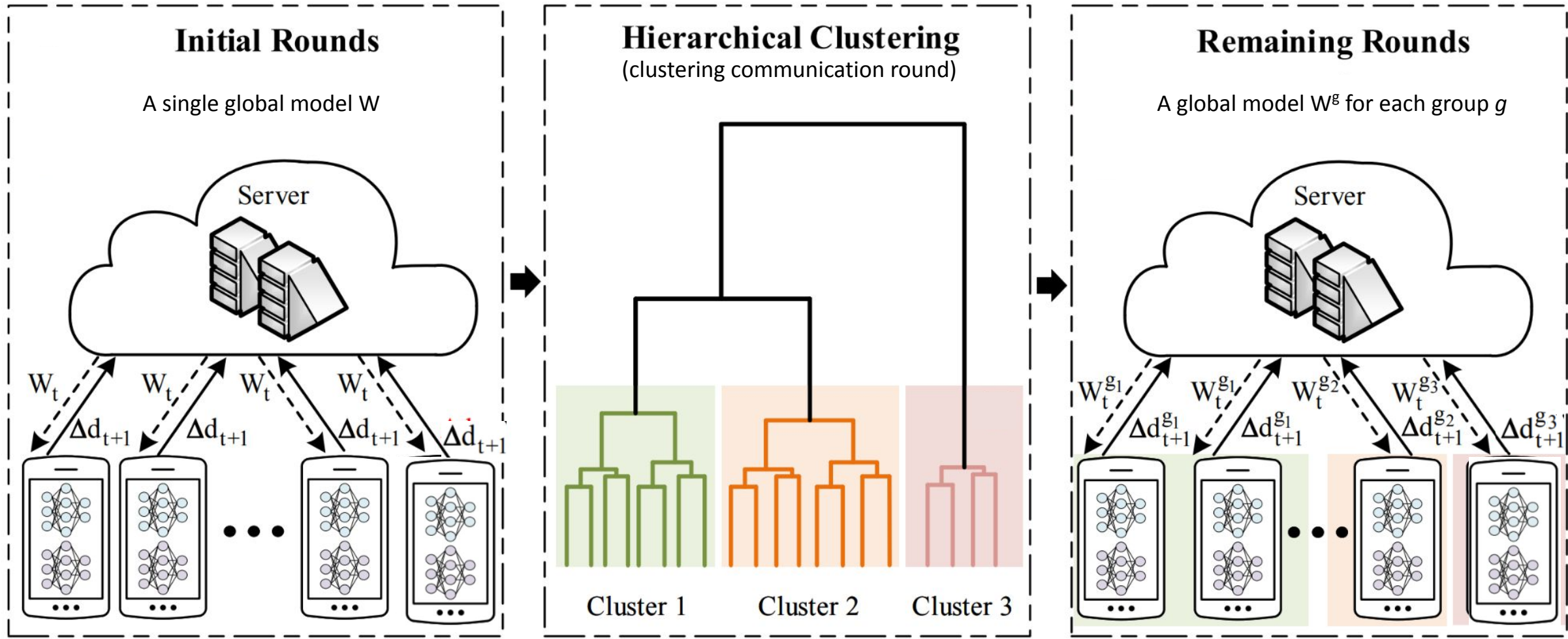
- **Idea of the algorithm:**

- compute ***pairwise similarity*** to group local model updates
- at the begin, there is one cluster for each user
- the algorithm finds the two clusters that are most similar
- when two clusters are grouped, a new specialized model is generated (e.g., using FedAvg)
- this process is repeated until convergence (based on a threshold on similarity)
- clusters with only one clients are considered as *non-clustered clients*

Federated Clustering - Server Side

- The main difference from standard Federated Learning is the algorithm computed by the server:
 - During the first $r-1$ communication rounds, a single global model is trained (no clustering)
 - At the r -th communication round, hierarchical clustering is applied to obtain specialized models
 - During the remaining communication rounds, the server updates the specialized global models

Federated Clustering Steps



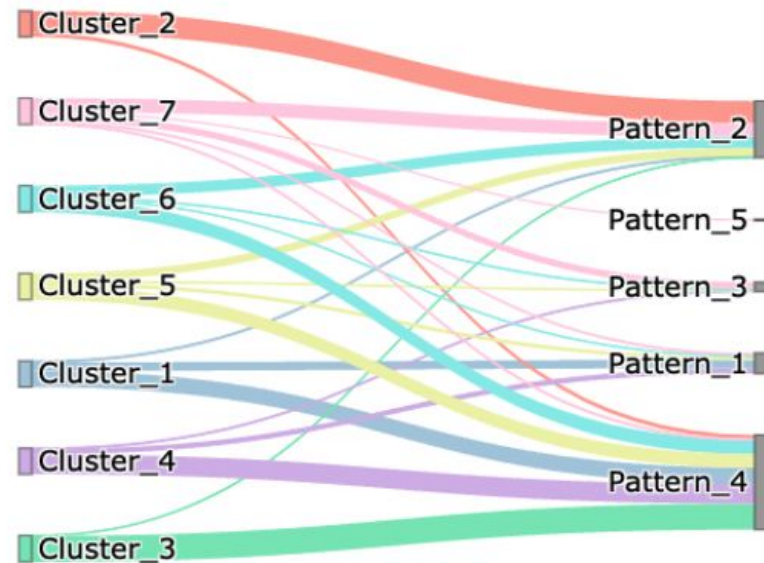
Clustering Round

- As described in the previous slides, clustering is performed by the server at a specific communication round
- The choice of the specific communication round at which performing clustering is a parameter that should be properly tuned:
 - if clustering is performed too soon, local models are not trained enough
 - if clustering is performed too late, the risk is that local models are too similar between them

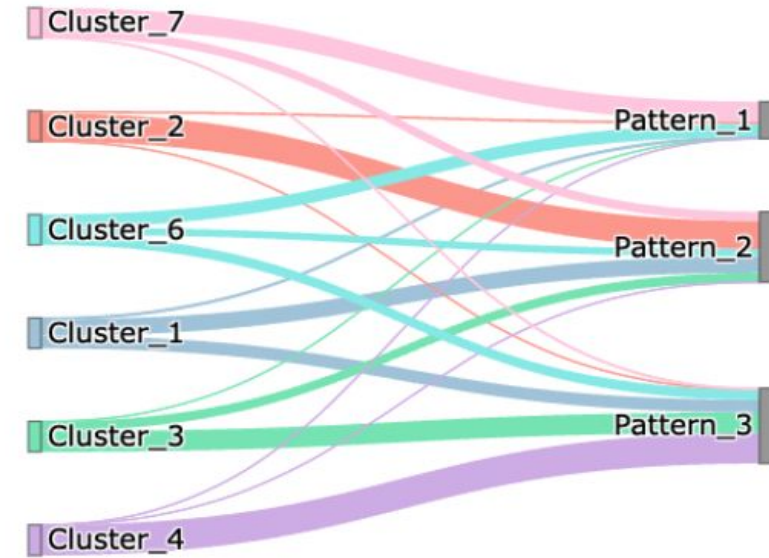
Clustering for feature distribution skew

Different groups of clients may exhibit different sensor data patterns.

Federated Clustering allows having specialized global models for users with similar characteristics.



(a) Going Downstairs

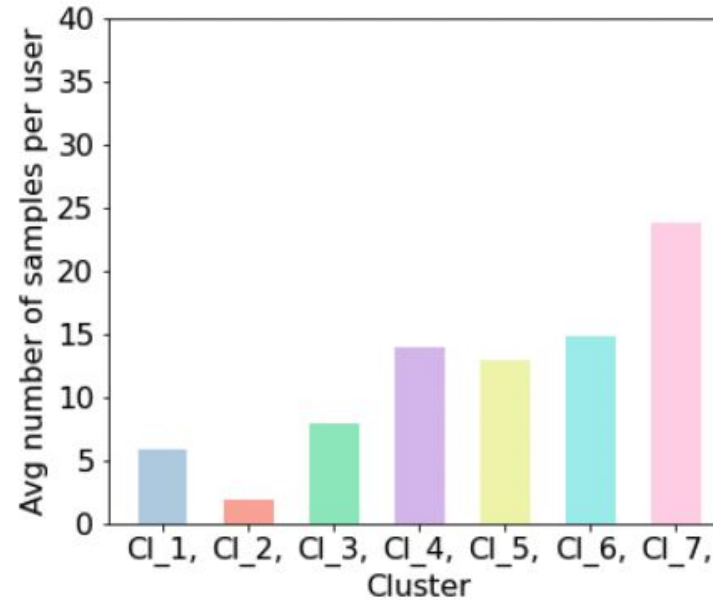


(b) Jogging

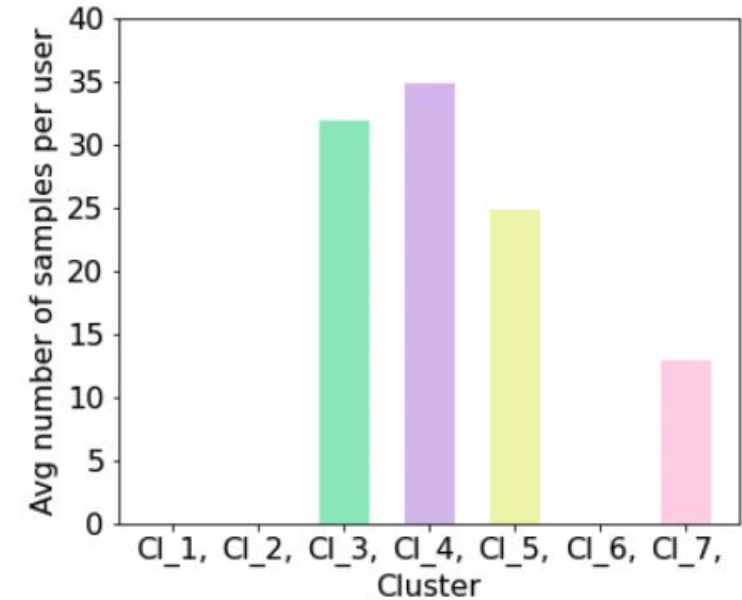
Clustering for label distribution skew

Different groups of users may have different label distributions (e.g., different habits).

Federated Clustering allows creating specialized global models for users with similar labels distribution.



(a) Going Upstairs



(b) Sitting

Labeled Data Scarcity in Federated Learning

Labeled data scarcity

- FL requires the participating clients having some labeled data to perform local training
- As extensively studied during this course, this assumption is not realistic in Ambient Intelligence scenarios
- In the following, we will study approaches to mitigate the data scarcity problem in FL

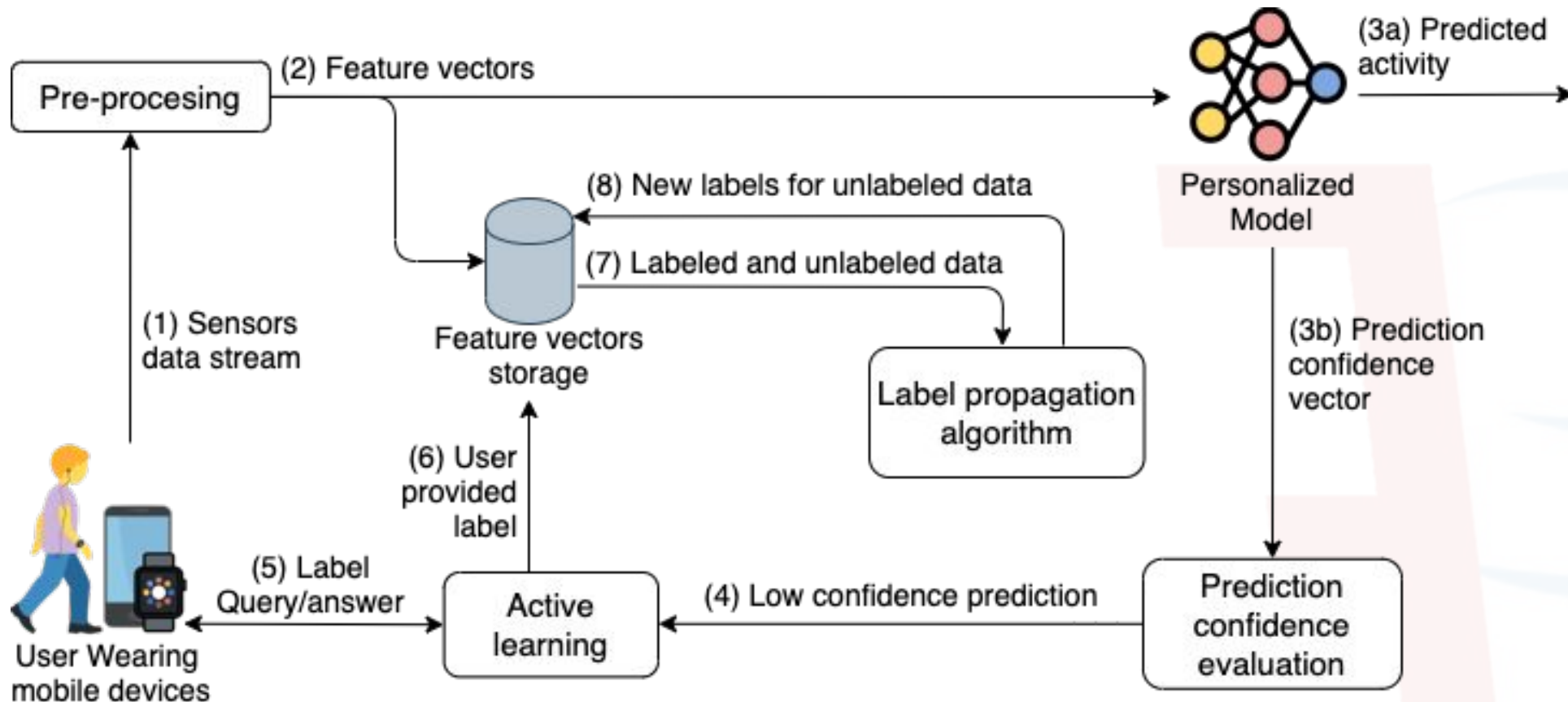
Semi-supervised Federated Learning

- The idea of semi-supervised learning is initializing the model with a small labeled dataset, so that it is possible to leverage unlabeled data to improve the model
- How to integrate this concept in FL?
 - the global model is initialized with a limited training set
 - at the begin, each client receives this pre-trained global model
 - locally, each client uses semi-supervised strategies to obtain pseudo labels
 - pseudo labels are then used to perform local training, so that local model parameters can be transmitted to the server

A possible solution: Combining Active Learning and Label Propagation

- A possible solution is to combine active learning and label propagation:
 - each client starts with a pre-trained model
 - each unlabeled sample is stored locally on the device
 - active learning is used to label a small number of unlabeled data points
 - periodically (e.g., at the end of each day) **label propagation** can be adopted to spread the labels acquired through active learning to a larger amount of unlabeled data points
 - the pseudo-labeled dataset can be used to train the local model!

Semi-supervised FL: Combining Active Learning and Label Propagation



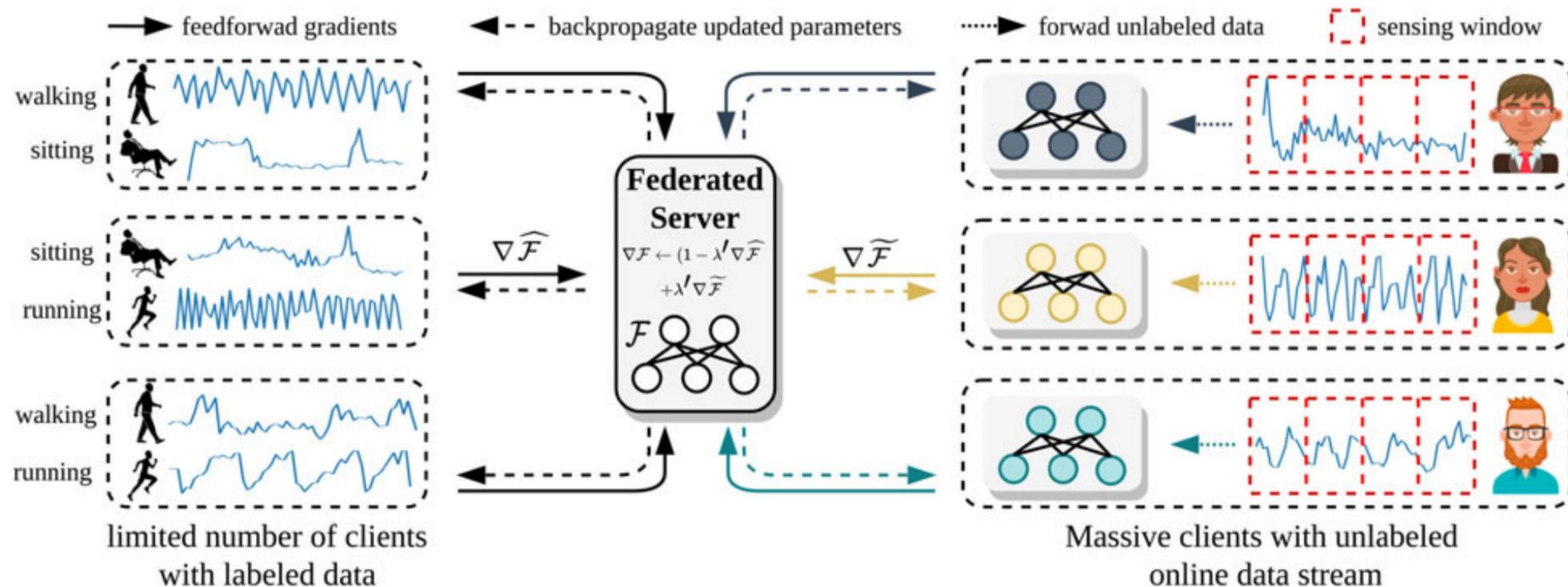
Presotto, R., Civitarese, G., & Bettini, C. (2022). Semi-supervised and personalized federated activity recognition based on active learning and label propagation. *Personal and Ubiquitous Computing*, 26(5), 1281-1298.

The feature vector storage will contain unlabeled and pseudo labeled data points. Periodically, pseudo labeled data can be used to perform local training. The weights of the resulting model can be used for FL.

Combining clients with labeled data and clients without labeled data

- It is not always possible for each client to rely on active learning
 - remember, it requires *interaction* with humans!
- However, sometimes it is possible to assume that there are clients with labels (or pseudo labels) availability, while other clients only have unlabeled data points
- In such cases, there are aggregation strategies to *transfer* knowledge from “*labeled clients*” to “*unlabeled clients*”
- Idea:
 - compute supervised model weights from labeled clients
 - compute unsupervised model weights from unlabeled clients (e.g., self-supervised learning)
 - aggregate the two types of models!

Combining clients with labeled data and clients without labeled data: the architecture

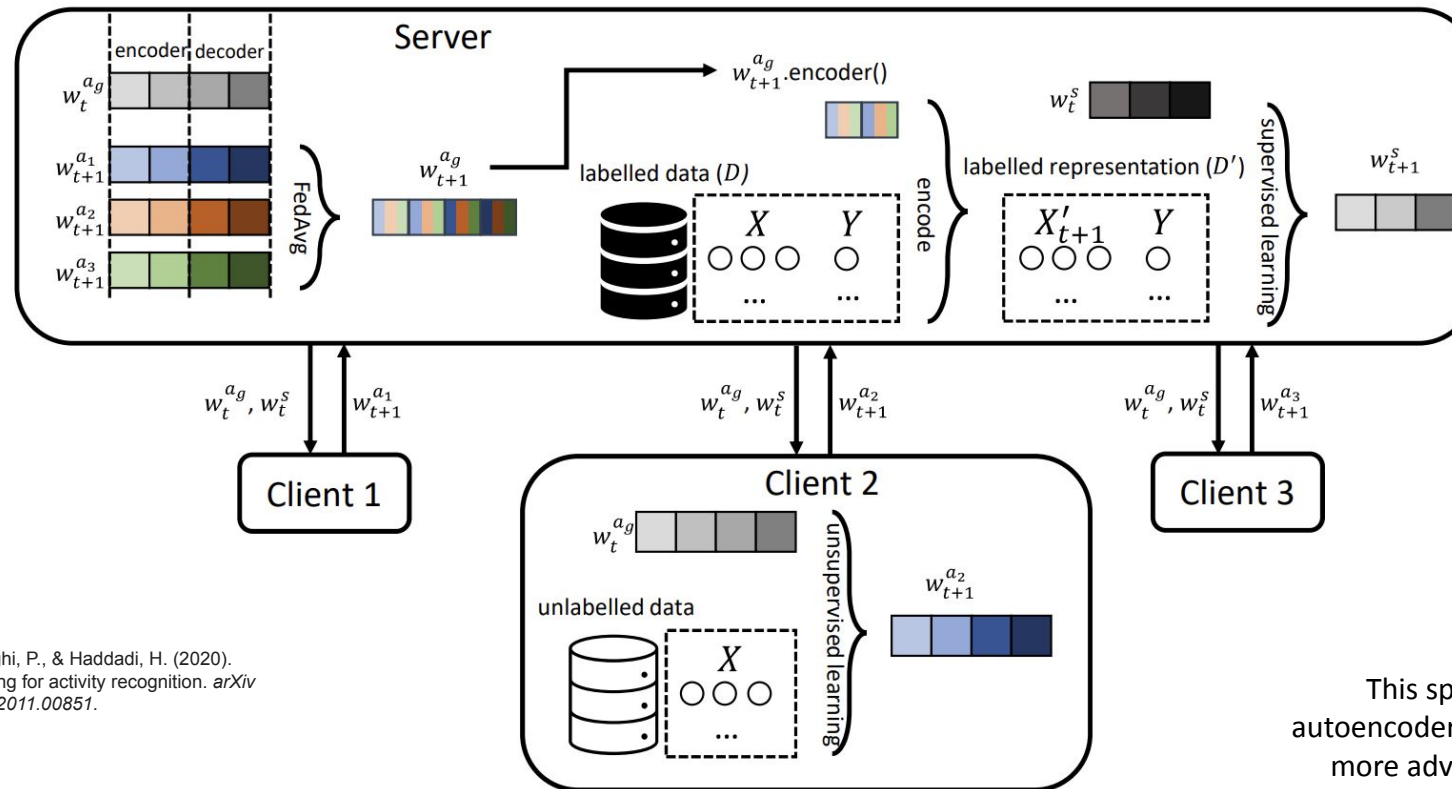


Yu, H., Chen, Z., Zhang, X., Chen, X., Zhuang, F., Xiong, H., & Cheng, X. (2021). Fedhar: Semi-supervised online learning for personalized federated human activity recognition. *IEEE Transactions on Mobile Computing*.

Self-supervised Federated Learning

- Worst-case scenario: there is no client with labeled data!
- Each client can leverage large amounts of unlabeled data to perform self-supervised learning locally (local feature representation)
- The server receives the locally trained feature extractors and aggregate them to generate a global self-supervised model
- Finally, the resulting global model can be fine-tuned with a small amount of labeled data
 - e.g., publicly available datasets

Self-supervised Federated Learning: overview



Zhao, Y., Liu, H., Li, H., Barnaghi, P., & Haddadi, H. (2020). Semi-supervised federated learning for activity recognition. *arXiv preprint arXiv:2011.00851*.

This specific solution is based on autoencoder for feature representation, but more advanced methods can be used.

Is Federated Learning really protecting privacy?

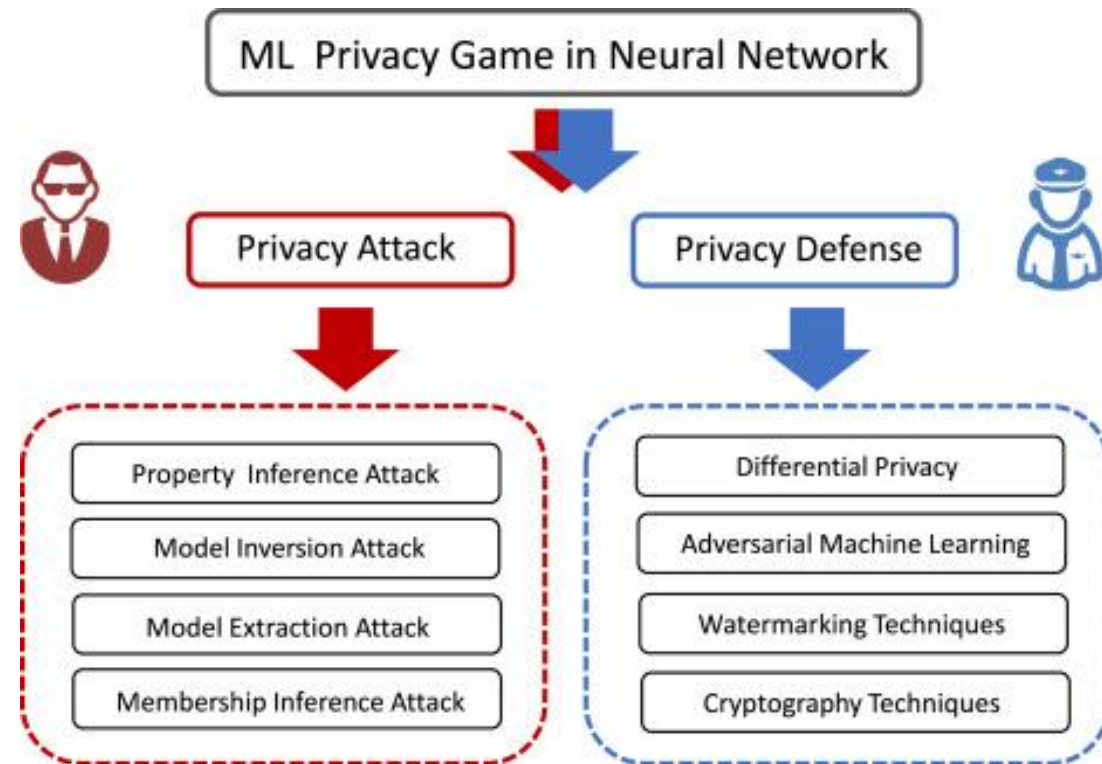
Privacy

- In FL, each client only shares model weights (and not data)
- This is a way to protect the privacy of the users
- However, this may not be enough!
- In general, the weights of a deep learning model can still leak information about the participating users
 - there are indeed several techniques to attack deep learning models (e.g., the global model learned by the server) in order to obtain some sensitive information!

Attacks and Defenses of Deep Learning Models

It is important to take countermeasures, so that the deployed ML models do not reveal sensitive information on the training data.

We will see some of the most known attacks to deep learning models that can lead to obtain sensitive information on the subjects that contributed with training examples



Attack 1: Model Inversion Attack

- The trained classifier can memorize information regarding the training dataset
- Goal of the attacker: deduce useful content about the target classifier's training data from the model
- The attacker only knows the target labels and wants to reconstruct typical samples of that class
- Particularly dangerous when classes represent particular individuals
 - indeed, it has been mainly considered for face recognition, where each class represents a specific subject!

Model Inversion: how it works

- Idea:
 - Provide random noise as input through the model that is being attacked (target model)
 - Backpropagate the loss
 - Instead of changing the weights, change the input.
 - Instead of optimizing the weights to minimize the loss, optimize the input to minimize the loss
 - i.e., generating the input that the model thinks is the most likely sample of a class!



Attack 2: Property Inference Attack

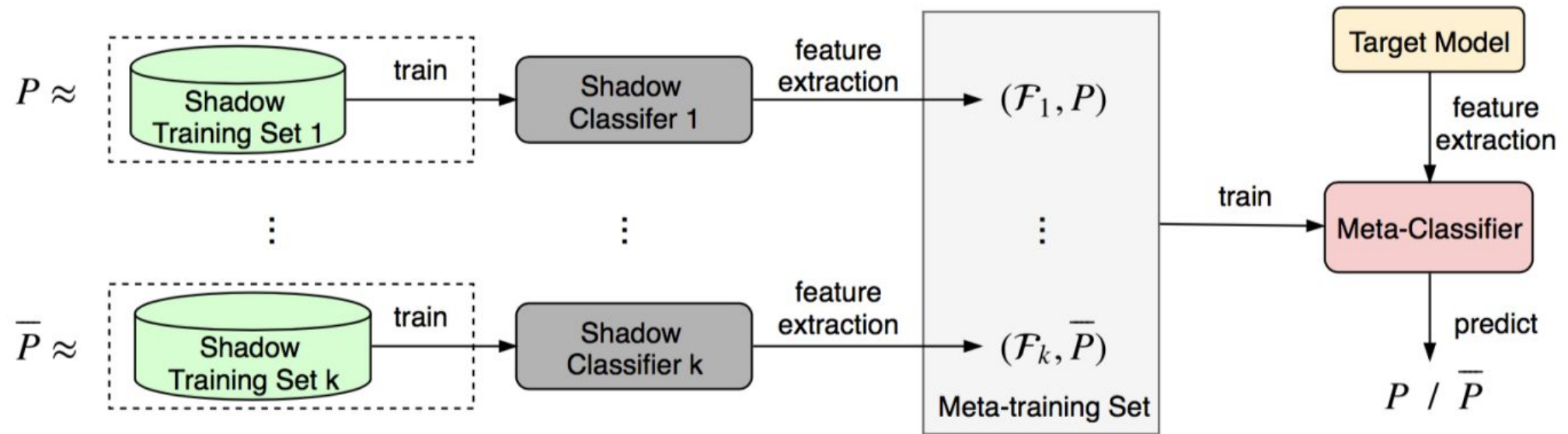
“Given only a model f , can an adversary infer some properties of the training dataset D or training algorithm A that the model owner does not intend to share?”

- Goal: extracting information that the model unintentionally learned, while they were not explicitly provided during training
 - *e.g., information about the ratio of men and women in a patient dataset, despite that gender information was not provided for the training records*
 - in Ambient Intelligence it may be possible to reveal sensitive health properties of the users from sensor data time series

Property Inference Attack: how does it work

- The attacker creates a set of ***shadow datasets***
- Each shadow dataset is a dataset with a distribution that is similar to the one used to train the target model, but constructed explicitly to either have or not have a property P.
- For each shadow dataset, a ***shadow model*** is trained
- Specific features are extracted from the shadow models, in order to train a ***meta-classifier***
 - it is a binary classifier that determines whether a model has the property P or not
- The resulting meta-classifier is used to attack the target model

Overview of Property Inference Attack



Attack 3:

The Membership Inference Attack

- Objective: infer whether a specific data sample has been used or not to train a target model (e.g., global FL model).
- A binary classifier (i.e., the **attack model**) is used to determine if a given sample is a member (or not) of the training set of the target model
- Two types of attacks:
 - **black box**: it is possible only to use the model to obtain predictions
 - **white box**: it is also possible to look inside the model (e.g., layers, parameters)

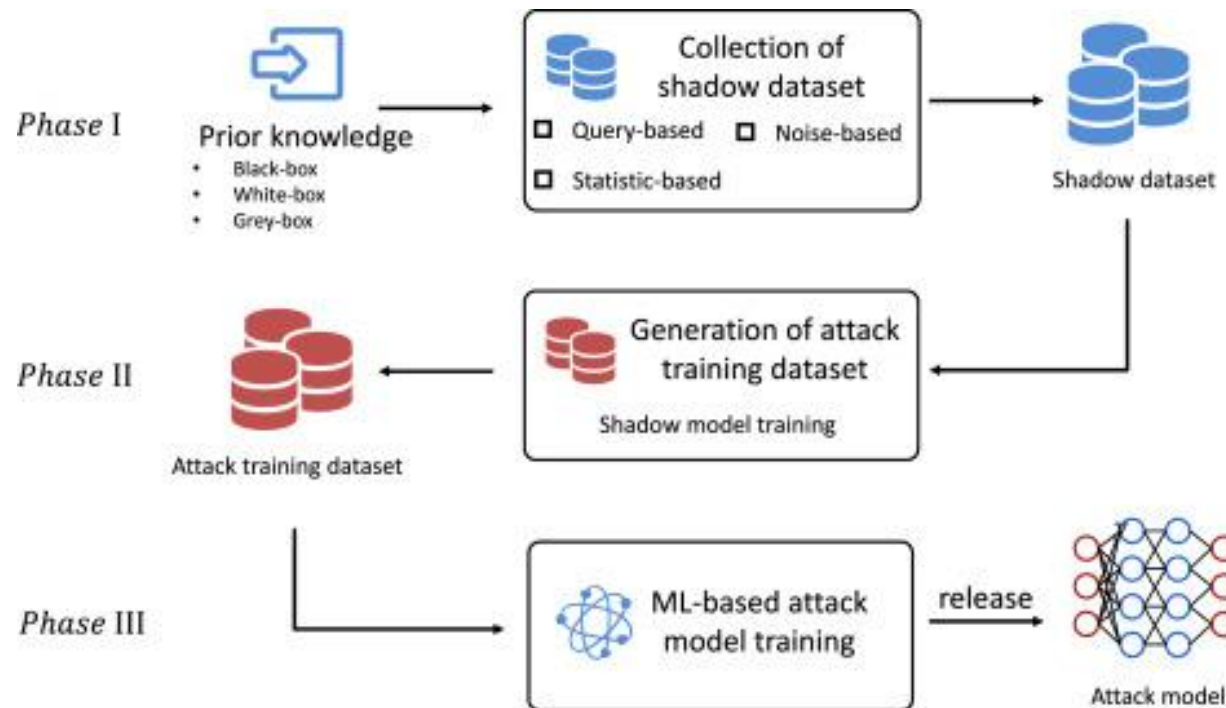


Training set

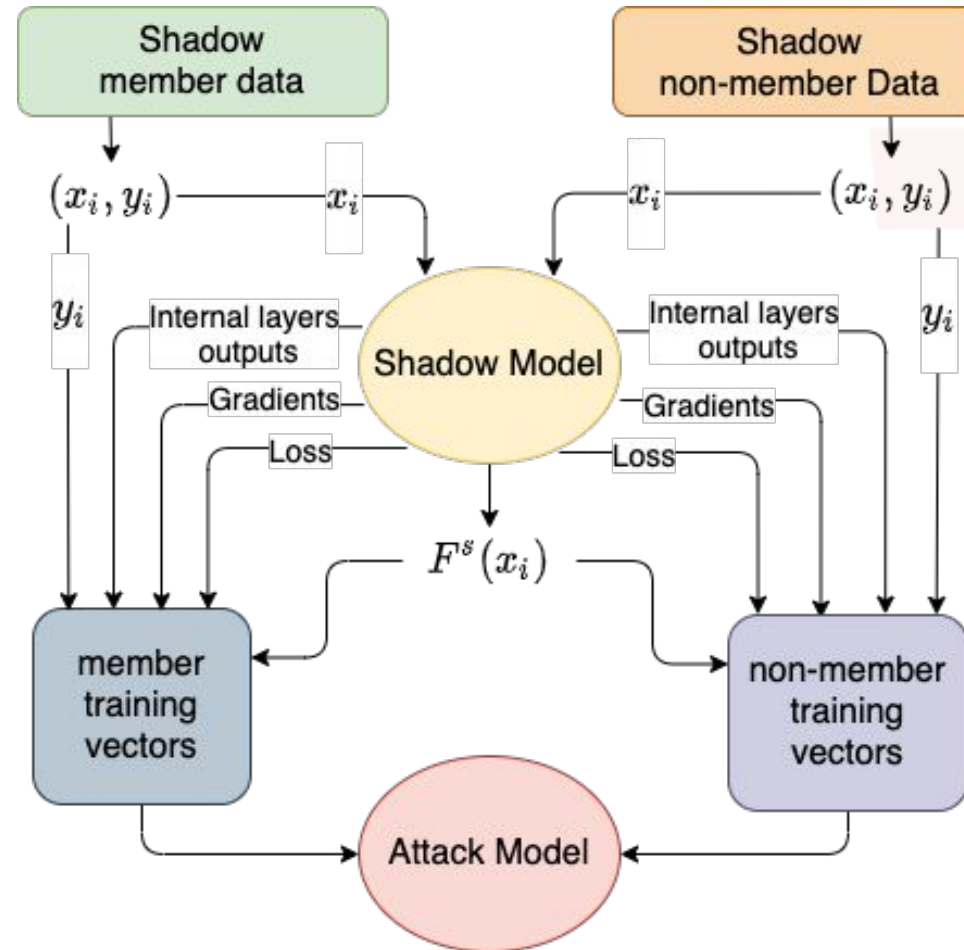
Shadow models

- The attacker has no access to the training dataset
 - it builds a **shadow model** that aims at imitating the behaviour of the target model (i.e., the one under attack, like the global FL model)
 - it creates a **shadow dataset** with
 - **member samples**: data with a similar distribution to the target training dataset (e.g., using similar data from other sources, generating synthetic data)
 - **non-member samples**: data that significantly differ from the target
- The attack model is trained by analysing the behaviour of the shadow model while classifying target samples
 - after training, the attack model is used to analyze the behavior of the target model to individuate members!

Membership Inference Attack phases



Training the attack model: a white box approach



The output of the attack model

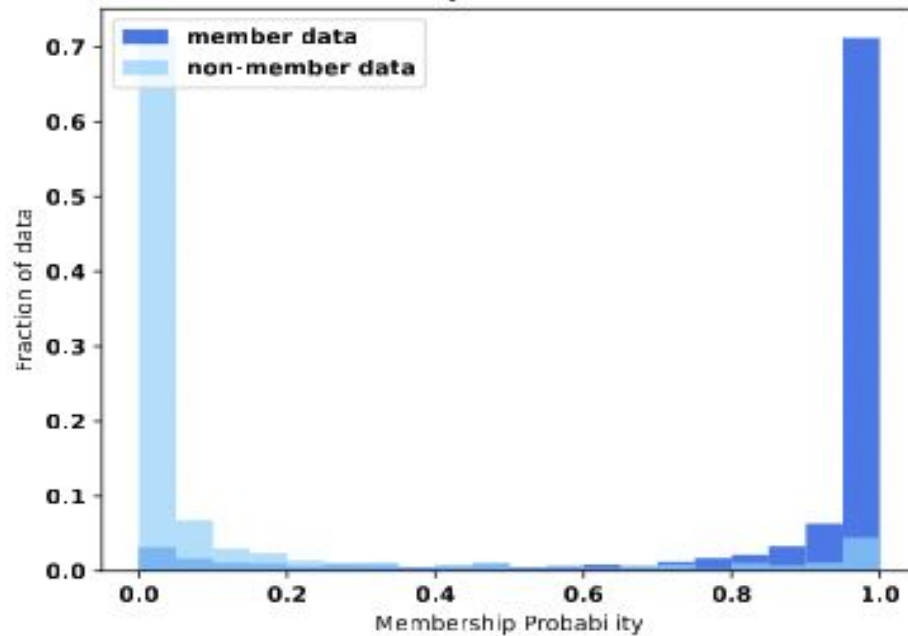
- To determine whether a data sample is a member or not, the Membership probability (MP) is used
 - values close to 1 indicates that the sample is likely a **member**
 - values close to 0 indicates that the sample is likely a **non-member**.
- Usually, MP is the output of the attack model (that is a binary classifier)

$$MP(x) = Pr(x \in D^t | F^t)$$

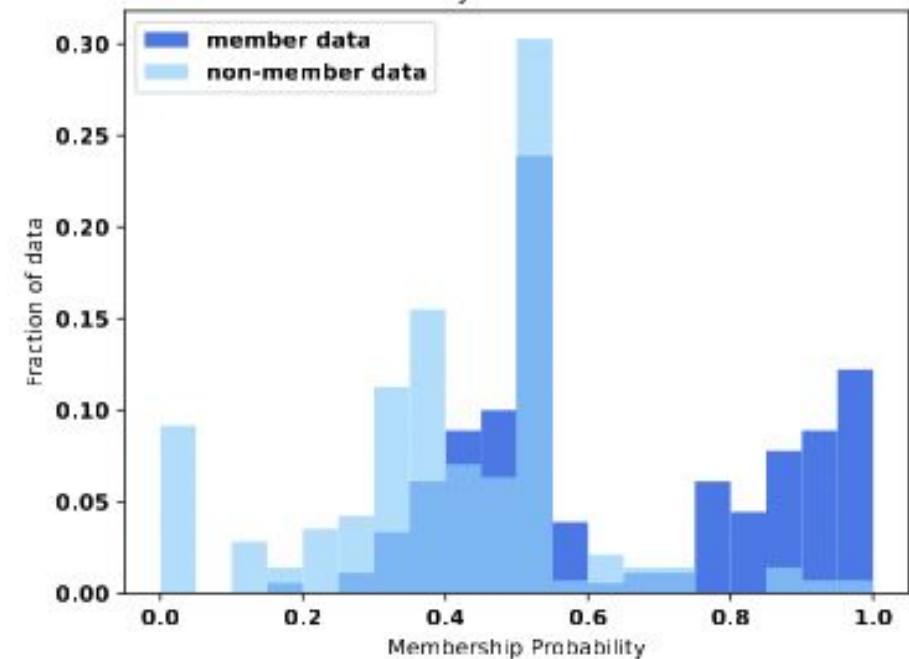
the probability of a sample belonging to the training dataset D^t given a model F^t

Membership Probability

an ideal case where the attack is successful



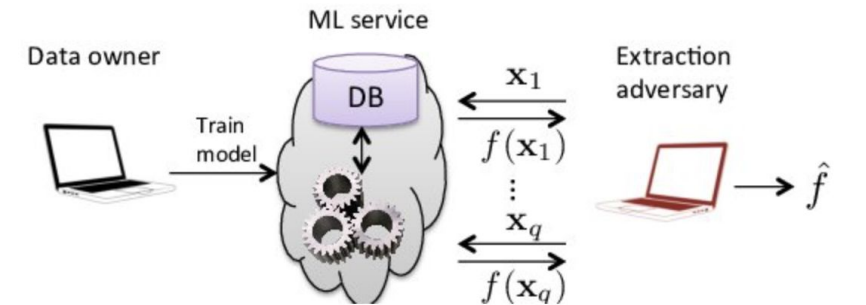
an example of unsuccessful attack



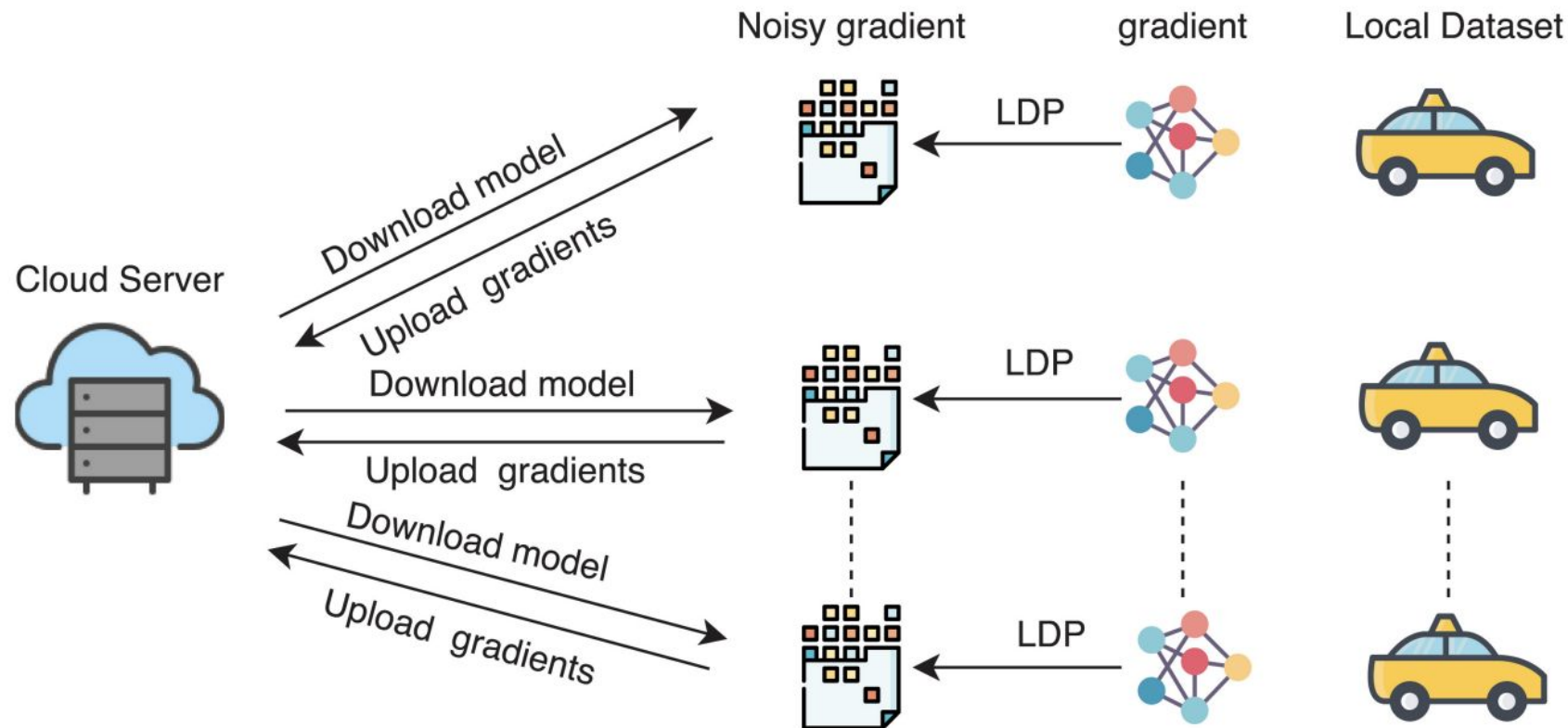
Evaluation carried out by matching the output of the attack model (i.e., Membership Probability) with the ground truth

Attack 4: Model Extraction Attack

- The adversary wants to reconstruct a black-box model f by creating a substitute model f'
 - f' should behave very similarly to f !
- Goal: obtaining a white-box version of the model, so that it is possible to attack it using other methods (e.g., model inversion, membership inference)
- No prior knowledge of model parameters and training data
- The goal is to reconstruct the model (e.g., with knowledge distillation) accurately with a low number of queries to the model
 - the model is usually reconstructed by using pairs of input and responses as a training data!



A possible countermeasure in FL: Local Differential Privacy (LDP)



Zhao, Y., Zhao, J., Yang, M., Wang, T., Wang, N., Lyu, L., ... & Lam, K. Y. (2020). Local differential privacy-based federated learning for internet of things. *IEEE Internet of Things Journal*, 8(11), 8836-8853.

References

- [1] Nguyen, D. C., Ding, M., Pathirana, P. N., Seneviratne, A., Li, J., & Poor, H. V. (2021). ***Federated learning for internet of things: A comprehensive survey***. IEEE Communications Surveys & Tutorials, 23(3), 1622-1658.
- [2] Ma, X., Zhu, J., Lin, Z., Chen, S., & Qin, Y. (2022). ***A state-of-the-art survey on solving non-IID data in Federated Learning***. Future Generation Computer Systems, 135, 244-258.
- [3] Ye, M., Fang, X., Du, B., Yuen, P. C., & Tao, D. (2023). ***Heterogeneous federated learning: State-of-the-art and research challenges***. ACM Computing Surveys, 56(3), 1-44.
- [4] Presotto, R., Civitarese, G., & Bettini, C. (2022, March). ***FedCLAR: Federated Clustering for Personalized Sensor-Based Human Activity Recognition***. In 2022 IEEE International Conference on Pervasive Computing and Communications (PerCom) (pp. 227-236). IEEE.